CAVER: Calibrated Active Verification for Real-Interaction Post-Training of Vision-Language-Action Policies

Interface-general method specification with concrete Stage-0 LIBERO and Stage-1 PiPER instantiations

March 2026

Abstract

This proposal studies one practical question in VLA post-training: when only a small number of real robot executions can be trusted, how should those executions be spent so that a policy improves reliably? Recent work shows that real post-training can help, world models can cheaply generate large numbers of imagined futures, and flow-based VLA backbones such as $\pi_{0.5}$ are attractive but awkward for standard policy-gradient RL because action likelihoods are not directly available in the usual way [1–4]. CAVER is therefore written at two levels. At the *method level*, it is interface-general: it assumes only (i) a deterministic execution adapter Γ_{exec} that maps raw backbone chunks into a safety-checkable execution space, (ii) a provider adapter Φ_Ω that maps execution chunks into the frozen provider's conditioning format, (iii) a deterministic safety shield S_{safe}, and (iv) a verified labeler M_{task} that maps an executed episode to a trusted outcome. At the *implementation level*, this proposal pins one staged study on the pretrained $\pi_{0.5}$ backbone from `openpi` and frozen GE-Sim from Genie Envisioner, with Stage 0 trusted execution in LIBERO simulation and Stage 1 real execution on PiPER [5, 6].

Within that general interface, the paper makes two more locked choices. First, the real-only update backend is a π-StepNFT-style executed-trajectory objective, not PPO or critic-heavy Q-learning [2]. Second, the selector architecture is fixed after Stage 0; only its utility inputs are refreshed round-by-round through a doubly robust calibrator. At each decision context, $\pi_{0.5}$ proposes K action chunks. A frozen provider predicts a short-horizon future for each chunk, and a small proxy head trained only on Stage-0 simulation and a seed real dataset produces a cheap utility estimate. A stochastic selector chooses one candidate for trusted execution from the safety-approved subset and logs its exact propensity. Every method shares the same deterministic pre-execution safety shield, so unsafe chunks are rejected before they can reach the trusted execution environment. The resulting trusted binary outcome is then converted into doubly robust pseudo-outcomes for *all safety-approved* candidates in that context using the logged propensity and the lagged nuisance model. A lagged calibrator converts cheap features into a corrected utility mean and scale. Only *executed and confident* trajectories are then passed to the post-training backend. Cheap imagined candidates never directly update the backbone in the main paper. Whenever the paper compares calibration across methods, it does so through a common post-hoc diagnostic predictor rather than through each method's native online confidence output.

The proposal is intentionally narrow. Its novelty claim is not that doubly robust estimation itself is new, and not that CAVER is a new end-to-end RL optimizer. The claim is that an *online selective-verification layer* — exact-propensity candidate selection, real-outcome correction, and conservative admission into an otherwise unchanged executed-sample backend — can make a fixed flow-based VLA post-training stack more sample-efficient under the same verified real budget. The primary comparisons are therefore: (i) real-only π-StepNFT on the same $\pi_{0.5}$ backbone, (ii) the world-model-heavy WoVR baseline through RLinf, and (iii) attribution-focused ablations under exactly the same real-rollout budget [4, 7]. The intended paper is a carefully controlled one-robot, three-task real-budget study with an interface-general method statement, not a claim about all robots or all RL backends.

Thesis in one sentence. Use one frozen provider for scale, use one small budget of real executions for truth, and place an interface-general selective-verification layer between them, with one fully

pinned Stage-0 LIBERO simulation instantiation and one fully pinned Stage-1 PiPER hardware instantiation in the implementation plan.

Locked design decisions

Decision point	Locked choice	Why it is locked
Action backbone	Pretrained $\pi_{0.5}$ from `openpi`	Matches the public flow-based VLA stack and supports a direct comparison with flow-based real-only post-training [8, 9].
Method presentation	Interface-general modules $(\Gamma_{\text{exec}}, \Phi_\Omega, S_{\text{safe}}, M_{\text{task}})$ plus concrete Stage-0 LIBERO and Stage-1 PiPER instantiations	Keeps the scientific claim at the interface level while preserving one exactly reproducible simulation-first path and one exactly reproducible hardware transfer study.
Cheap future provider	Frozen GE-Sim from Genie Envisioner	Satisfies the preference for an off-the-shelf world model and keeps the paper about verification rather than training a new simulator [5, 6].
Concrete trusted-execution stacks	LIBERO simulation in Stage 0; PiPER single-arm manipulation in Stage 1	Makes the debugging and calibration stage concrete before the real study while keeping the eventual hardware transfer equally auditable.
Cheap utility head	Small 3-head MLP trained only on Stage 0 + seed real data	Avoids training a world model from scratch while still providing a task-specific scalar for selection.
Real-only backend	π-StepNFT-style executed-trajectory fine-tuning	Fits flow-based VLAs without requiring tractable action likelihoods or a learned critic [2, 10].
Execution safety	Shared deterministic safety shield plus runtime abort rules	Prevents unsafe candidate chunks from reaching hardware and makes the real study auditable across methods.
Selector learning	No online selector-parameter learning in the main method	Keeps the verification rule fixed and auditable. The selector form and coefficients are frozen after Stage 0; only the utility inputs refresh.
Frozen features	Provider and calibration features come from frozen encoders	Keeps the cheap-feature space fixed while policy adaptation touches only LoRA modules in the action head.
World-model baseline	WoVR via RLinf	Gives the strongest world-model-heavy comparison already supported in a public RL stack [4, 11].

1 Introduction

World models, verifier scores, and large VLA backbones have made it easy to generate many candidate robot behaviors together with cheap predicted futures and cheap reward-like signals. At the same time, recent post-training work keeps returning to the same bottleneck: the hard part is not producing *more* feedback, but producing feedback that is trustworthy enough to update the policy [1, 4, 12]. World-VLA-Loop, WoVR, TGRPO, NORA-1.5, and VLA-RFT all illustrate versions of the same tension. Learned simulators and reward proxies can provide scale, but simulator hallucination, compounding error, sparse success labels, and proxy exploitation make naive optimization brittle [4, 13–16].

This proposal takes that tension as the central problem. Let M be a large pool of cheap imagined or proxy-scored candidates, and let $N \ll M$ be a much smaller set of verified real robot executions. The scientific question is not "which reward function should we use?" It is: *how should a small real verification budget be allocated and converted into a reliable policy-improvement signal?*

The theory in this proposal is deliberately narrow. It justifies a *context-level* verification rule conditional on the stream of decision contexts that actually occurs; it does not claim a full episodic policy-improvement guarantee

after online backbone updates.

This question is especially sharp for flow-based VLAs. The current public `openpi` implementation supports $\pi_{0.5}$ with a flow-matching action head. Flow-based action generation makes standard likelihood-ratio RL awkward, critic learning can be heavy, and a practical first-paper route is a critic-free, likelihood-free executed-trajectory update such as π-StepNFT [2, 9]. Accordingly, the method keeps the backbone fixed at $\pi_{0.5}$ and keeps the executed-update backend close to that path rather than turning the proposal into a new flow-RL optimizer paper.

A second key design choice is to treat the world model as a *frozen provider* rather than a new training target. The mainline method therefore uses a frozen off-the-shelf provider, with GE-Sim from Genie Envisioner as the primary choice because the public repository exposes code, released weights for GE-Sim, and example inference scripts that consume images, camera intrinsics/extrinsics, and action arrays [5, 6]. This keeps the contribution focused on verification and makes the approach transferable across backbones and world models.

Finally, the proposal is positioned to be publishable because the claim is deliberately modest. CAVER is not presented as a new end-to-end RL algorithm for all flow-based VLAs. It is a *training-time selective-verification layer*. Relative to ALOE, which studies action-level off-policy evaluation from logged VLA data, CAVER adds an online data-collection decision: it chooses which cheap candidate to verify in the first place, then uses that verified outcome to decide which executed trajectories deserve an update [17]. Relative to WoVR and World-VLA-Loop, CAVER does not ask the world model to become increasingly reliable through policy/world co-training; it instead treats the world model as fallible and spends real interaction only where it is most valuable [4, 13]. Relative to real-only post-training, it asks whether the same real budget can be used more intelligently. That is a concrete, testable, and publishable contribution.

Contributions.

1. A **problem formulation** for VLA post-training under limited real verification, in which a frozen off-the-shelf world-model provider produces cheap predicted futures while real execution supplies the trusted label.
2. A **two-layer CAVER specification**: an interface-general method defined by an execution adapter, provider adapter, safety shield, and verified labeler, together with concrete Stage-0 LIBERO and Stage-1 PiPER instantiations.
3. A **roundwise selective-verification algorithm** with exact feature definitions, exact selector behavior, exact roundwise data flow, explicit frozen-vs-trainable components, and an implemented-path update backend.
4. A **reproducible implementation and evaluation plan** using $\pi_{0.5}$, GE-Sim, PiPER, LIBERO, π-StepNFT, and WoVR/RLinf, with matched-budget baselines and a clear contingency plan if the primary provider integration stalls.

Positioning against the nearest neighboring methods

What CAVER does relative to the closest neighboring VLA post-training papers

Method	Main object of optimization	How real data enters	What CAVER keeps or changes
ALOE	Action-level off-policy evaluation / value estimation from logged VLA chunks [17]	Logged executed data; no online verification-allocation rule	CAVER adds an online decision about *which* cheap candidate to verify before execution, then uses the verified outcome to decide which executed trajectories deserve admission to training.
π-StepNFT	Executed-trajectory post-training for flow-based VLAs without a critic or action likelihoods [2]	Uses executed trajectories directly	CAVER keeps this backend fixed in the headline comparison and asks whether selective verification can feed it more informative executed data under the same real budget.
πRL	RL backend for flow-based VLAs via Flow-Noise / Flow-SDE [3]	Large online interaction or simulation for policy-gradient style RL	CAVER is not a replacement for πRL; it intentionally stays on the simpler executed-sample backend and targets minimal-verification post-training rather than a new RL optimizer.
WoVR	World-model RL with imagined rollouts and hallucination control [4]	Real interaction plus many imagined rollouts	CAVER treats the world model as a frozen, fallible provider and asks how scarce verified real executions should be allocated, rather than how to optimize more aggressively inside the simulator.

2 Scope and explicit design choices

This section states the design choices that make the proposal both general at the method level and concrete at the implementation level.

2.1 What this paper is and is not

This is a **selective-verification paper**. It is not a full policy-gradient method for flow-based VLAs, it is not a new critic-learning paper, and it is not a world-model-training paper. The main contribution is also not doubly robust estimation by itself; that machinery is borrowed. The contribution claimed here is the *integration*: exact-propensity online verification allocation plus conservative admission into an unchanged executed-sample backend under a strict matched-budget protocol. The paper asks whether a small verified real budget can be used to obtain better learning signal than either direct real-only post-training or direct imagined-experience training. Natural extensions such as a πRL-style Flow-SDE policy-gradient backend, learned online selector policies, or multi-provider disagreement modeling are future work rather than part of the present claim.

2.2 What is general and what is instantiated

CAVER's scientific claim is not tied to PiPER-specific joint commands. At the method level, the pipeline is defined by four interfaces:

$$\tilde{a}^{\pi} \xrightarrow{\Gamma_{\text{exec}}} a \xrightarrow{S_{\text{safe}}} \nu_r \xrightarrow{\text{execute}} \text{episode trace} \xrightarrow{M_{\text{task}}} Y, \qquad a \xrightarrow{\Phi_{\Omega}} \tilde{a}^{\Omega} \xrightarrow{\mathcal{T}_{\Omega}} \hat{I} \xrightarrow{V_{\omega}, f_{\psi}} (\hat{v}, \hat{\sigma}, \mu, \sigma).$$

Only four modules are robot- or simulator-specific: the execution adapter Γ_{exec}, the safety shield S_{safe}, the provider adapter Φ_{Ω}, and the verified labeler M_{task}. The staged study instantiates them twice. In Stage 0, LIBERO supplies simulator execution, simulation-side candidate validity checks, a LIBERO-to-GE-Sim compatibility map, and the simulator task-success labeler. In Stage 1, PiPER supplies joint/gripper execution, a PiPER URDF-based safety shield, a PiPER-to-GE-Sim compatibility map, and a marker-based geometric labeler. Porting CAVER to another robot changes those modules and the local configs, but it does *not* change

selector scoring, exact propensity logging, doubly robust correction, calibrator fitting, or the StepNFT-style executed-update backend.

2.3 Why the mainline uses an off-the-shelf world model

The mainline method uses a frozen off-the-shelf provider because this is the cleanest way to isolate the proposed contribution. A new world model would introduce another training problem, another set of hyperparameters, and another source of failure. GE-Sim is therefore the primary provider because its public repository includes released GE-Sim weights, an example inference path, and explicit instructions for passing images, camera intrinsics/extrinsics, and actions into the simulator [6]. The proposal also keeps a local lightweight provider only as a *contingency*; see Appendix D. Replacing GE-Sim in a future study would change Φ_Ω and the provider-specific input bundle, not the selective-verification logic itself. The paper therefore generalizes across interfaces, not by claiming that every robot stack is already validated empirically.

2.4 Why the update backend is executed-sample π-StepNFT

Critic-free, likelihood-free fine-tuning is the most direct practical route on $\pi_{0.5}$, and the public ecosystem supports that route. The open-source π-StepNFT code is public and is already framed as a critic-and-likelihood-free method for flow-based VLAs [2, 10]. This proposal therefore uses the released π-StepNFT objective as the real-only baseline and as the backbone-update backend inside CAVER. In the main comparison, CAVER keeps the StepNFT loss, optimizer, LoRA target modules, batch size, gradient clip, and update schedule unchanged. It differs only in how candidate chunks are scored before execution, which executed contexts are admitted to training, and which exact propensities are logged for doubly robust correction.

2.5 How the selector is fixed yet still improves over rounds

The **selector functional form is frozen after Stage 0**. Its scalar coefficients, temperature, and exploration floor are tuned once in Stage 0 and never learned online. However, its value and uncertainty inputs are allowed to refresh round-by-round through the latest calibrated utility model. In other words, the selector does *not* have trainable online parameters, but it can still make better decisions over time because it reads the newest corrected utility estimates.

2.6 Frozen features and trainable components

The cheap-feature space does **not** move while the policy is being updated. The encoders used to build $e_{r,c}$, $q_{r,c,k}$, and therefore $z_{r,c,k}$ are frozen throughout Stage 1. Only LoRA modules are updated inside the action head used for the final policy improvement. Here *LoRA* means *low-rank adaptation*: small trainable low-rank matrices are inserted into a frozen pretrained network so that most original weights remain fixed.

3 Problem setup, notation, and repeated terms

3.1 Roundwise decision process

Training proceeds in rounds $r = 1, 2, \ldots, R$. **Each Stage-0 and Stage-1 online training run is task-specific**: one run fine-tunes one copy of the common seed-warmed checkpoint on exactly one task, and all reported task averages are formed only after those per-task runs finish. A **decision context** is a chunk boundary inside an episode, not an individual low-level control step. Each episode allows at most four decision contexts.

A **round** is operationally fixed to

$$B_{\text{round}} = 25$$

counted training contexts on a single task. A counted context is either (i) one safety-approved candidate that is actually executed on hardware or in simulation, or (ii) one "no safe candidate" event that is logged as

`safety_abort` and charged to the budget even though no motion is executed. Because all main budgets satisfy $N \in \{25, 50, 100, 200\}$, every run has exactly

$$R = N/B_{\text{round}} \in \{1, 2, 4, 8\}$$

rounds and there is no partial final round in the main experiments. Within one round, the backbone policy π_{θ_r}, the selector ν_r, the lagged nuisance model m_{r-1}, and the safety shield are frozen. Only after the round closes are the calibrator and, if triggered, the LoRA modules updated.

This roundwise convention makes the theory–protocol link explicit. The potential outcome $Y_{r,c}(k) \in \{0, 1\}$ means the eventual episode-success label that would be observed in round r if candidate k were executed at context c and the remainder of the episode were completed by the frozen continuation policy π_{θ_r}. The observed label is $Y_{r,c} = Y_{r,c}(A_{r,c})$.

3.2 Method-level interfaces and current instantiation

The method itself is written around two deterministic translation steps and two deterministic trust layers:

$$\tilde{a}^{\pi}_{r,c,k} \xrightarrow{\Gamma_{\text{exec}}} a_{r,c,k} \xrightarrow{S_{\text{safe}}} \text{trusted execution}, \qquad a_{r,c,k} \xrightarrow{\Phi_\Omega} \tilde{a}^{\Omega}_{r,c,k} \xrightarrow{\mathcal{T}_\Omega} \hat{I}_{r,c,k,1:H}.$$

The first map, Γ_{exec}, turns raw backbone outputs into a safety-checkable execution-space chunk for the chosen trusted execution environment. The second map, Φ_Ω, turns that same execution chunk into the frozen provider's conditioning format. The trust layers are (i) the hard safety shield S_{safe}, which determines which candidates may be executed at all, and (ii) the verified labeler M_{task}, which turns the executed episode into a trusted binary outcome. In the present study, Stage 0 instantiates these modules with a LIBERO execution adapter, simulation-side validity checks, a LIBERO-to-GE-Sim compatibility map, and the simulator task labeler; Stage 1 instantiates them with a PiPER joint/gripper adapter, a PiPER-to-GE-Sim compatibility map, a PiPER URDF-based hard filter, and a marker-based geometric labeler. The selector, DR correction, calibrator, and StepNFT-style update backend do not depend on which concrete robot or simulator instantiates those four modules.

3.3 Master notation table

Table 1: Master notation table used throughout the proposal.

Symbol	Name	Exact meaning in this proposal
r	round index	One online collection-and-update block of exactly $B_{\text{round}} = 25$ counted training contexts on a single task.
c	context index	Decision-context index within the current round and episode.
B_{round}	round size	Fixed at 25 counted training contexts per round in both Stage 0 and Stage 1.
$h_{r,c}$	observation bundle	Policy-view observations, optional provider-only auxiliary views, robot state, and any calibration metadata required by the concrete instantiation.
$g_{r,c}$	instruction	Task-language string for the current episode.
$s^{\text{prop}}_{r,c}$	robot state	Current robot state passed to the policy and execution adapter; in the current study this is joint position/velocity plus gripper state.
$e_{r,c}$	frozen context embedding	Context embedding produced by the frozen encoder E^{frz}_ϕ.
E^{frz}_ϕ	frozen context encoder	The frozen $\pi_{0.5}$ context encoder used to form $e_{r,c}$ and start-state embeddings.
π_{θ_r}	backbone policy	Frozen $\pi_{0.5}$ backbone used during data collection of round r.
θ_{roll}	rollout checkpoint	Frozen policy parameters that originally generated the executed trajectories currently being optimized by StepNFT.
K	candidates per context	Fixed at $K = 4$ in all main experiments.
H	chunk horizon	Fixed at $H = 4$ executed commands per candidate chunk.
Δt	control interval	Fixed at $\Delta t = 0.1$ s, i.e. 10 Hz chunk execution.
d_{exec}	execution dimension	Per-step dimension of one execution-space command after Γ_{exec}; the current study uses $d_{\text{exec}} = 7$.

Symbol	Name	Exact meaning in this proposal
d_Ω	provider dimension	Per-step dimension of one provider-conditioning vector after Φ_Ω; the current study uses $d_\Omega = 16$.
$\tilde{a}^{\pi}_{r,c,k}$	raw backbone chunk	The kth sampled raw chunk from π_{θ_r} before robot-specific execution conversion.
$a_{r,c,k}$	execution-space chunk	The kth $H \times d_{\text{exec}}$ chunk after Γ_{exec}; this is the command actually handed to the safety shield and, if selected, to the robot bridge.
$\tilde{a}^{\Omega}_{r,c,k}$	provider-space chunk	The $H \times d_\Omega$ chunk passed to the frozen provider after Φ_Ω.
Γ_{exec}	execution adapter	Deterministic map from raw backbone outputs and current robot state into safety-checkable execution commands; the current-study alias is $\Gamma_{\pi\to p}$.
Φ_Ω	provider adapter	Deterministic map from execution-space chunks into the provider's input action format; the current-study alias is Φ_{GE}.
S_{safe}	safety shield	Deterministic hard constraint checker applied before execution.
M_{task}	verified labeler	Deterministic task-success measurement rule applied after executed episodes.
$\Gamma_{\pi\to p}$	current-study execution adapter	Concrete PiPER instantiation of Γ_{exec}.
Φ_{GE}	current-study provider adapter	Concrete GE-Sim instantiation of Φ_Ω.
$\tilde{a}^{\text{GE}}_{r,c,k}$	current-study provider chunk	GE-Sim action array written by the present PiPER-to-GE-Sim compatibility map.
$\hat{I}_{r,c,k,1:H}$	predicted future outputs	The frozen provider's predicted outputs for candidate k across the H-step horizon.
$\hat{I}^{\text{head}}_{r,c,k,H}$	current-study summary frame	Final predicted head-view frame used as the cheap future summary in the present GE-Sim instantiation.
F^{frz}_ρ	frozen future encoder	Frozen visual encoder applied to the summary provider output.
R	random projection	One fixed Gaussian projection that maps the pooled future-frame feature to 64 dimensions.
$q_{r,c,k}$	future embedding	64-dimensional projected feature extracted from the provider summary output.
$\phi_a(\cdot)$	action encoder	Flattened standardized action representation defined in Step 3.
$V^{(b)}_\omega$	bootstrap head	The bth head of the 3-head cheap utility predictor.
$\hat{v}_{r,c,k}$	raw proxy utility	Mean cheap scalar utility estimate across the 3 bootstrap heads.
$\hat{\sigma}^{\text{pre}}_{r,c,k}$	proxy uncertainty	Standard deviation across the bootstrap heads of V_ω.
$\mathcal{D}_{\text{seed}}$	warm-start seed set	The fixed fully verified set used before round 1 to create the common seed-warmed checkpoint and fit CAVER's initial calibrator f_{ψ_0}.
f_{ψ_0}	initial calibrator	Calibrator fit once on $\mathcal{D}_{\text{seed}}$ before round 1; later rounds refit f_{ψ_r}.
$\mathcal{I}_V$	proxy-training index set	All Stage-0 train-split tuples plus the shared seed-real tuples used to fit the utility head.
$\mathcal{T}^{\text{seed}}_{\text{S0}}, \mathcal{T}^{\text{train}}_{\text{S0}}, \mathcal{T}^{\text{val}}_{\text{S0}}, \mathcal{T}^{\text{test}}_{\text{S0}}$	Stage-0 partitions	Disjoint Stage-0 seed, training, validation, and audit-test template lists defined once before any tuning.
J_{S0}	selector-selection score	Scalar Stage-0 model-selection score used only to choose frozen selector coefficients.
$b_{r,c,k}$	base feature vector	Cheap feature vector before novelty is appended.
$\mathcal{B}_{\text{hist}}$	executed-history reservoir	FIFO reservoir of executed base features used for novelty.
$D_{r,c,k}$	within-context disagreement	Mean action-space distance from candidate k to the other $K-1$ candidates in the same context.
$N_{r,c,k}$	novelty	Nearest-neighbor distance from $b_{r,c,k}$ to the executed-history reservoir.
$z_{r,c,k}$	final cheap feature vector	$z_{r,c,k} = [b_{r,c,k}, N_{r,c,k}]$.
$s_{r,c,k}$	selector score	Pre-softmax frozen-form selector score for candidate k in context (r, c).
$\nu_r(k \mid c)$	verification selector	Frozen-form stochastic selector over the K candidates in round r.
$p_{r,c,k}$	propensity	Exact logged selection probability $\nu_r(k \mid c)$.
$\mathcal{K}^{\text{safe}}_{r,c}$	safe candidate set	Indices of candidates that pass the deterministic pre-execution safety shield.
$m_{r-1}(z)$	lagged nuisance model	Cheap utility estimate available before round r begins.
$\tilde{y}^{\text{DR}}_{r,c,k}$	DR pseudo-outcome	Doubly robust corrected utility target for candidate k.
$\mathcal{D}^{\text{tuple}}_{\le r}$	accumulated tuple set	All non-aborted candidate tuples from verified contexts up to and including round r.
f_{ψ_r}	calibrator	Mean/scale predictor refitted after collecting round-r real data.
$\mu_r(z), \sigma_r(z)$	corrected utility	The calibrator's corrected mean and heteroscedastic scale.
$\bar{\mu}_r(z)$	clipped probability diagnostic	$\bar{\mu}_r(z) = \text{clip}(\mu_r(z), 0, 1)$ used only for probability-style metrics.
$\text{LCB}_r(z)$	conservative utility	$\mu_r(z) - \kappa\sigma_r(z)$.

Symbol	Name	Exact meaning in this proposal
$\mathcal{M}^{+}_{\le r}, \mathcal{M}^{-}_{\le r}, \mathcal{M}^{0}_{\le r}$	cumulative buffers	Positive, negative, and abstain executed-trajectory buffers used from rounds $1{:}r$.
$\mathcal{P}_{\le r}$	cumulative pair set	Positive–negative trajectory pairs built from the cumulative buffers after round r.
$z^{\text{exec}}_{r,c}$	evaluation feature	Method-agnostic executed feature vector used by the common post-hoc calibration predictor.
g_{eval}	diagnostic predictor	Common post-hoc probability predictor used only to compute cross-method Brier/ECE.
$N^{\text{ctx}}_{\text{val}}, N^{\text{ctx}}_{\text{test}}$	evaluation interaction counts	Real decision-context counts consumed by validation and test episodes.
N_{total}	total real interaction count	$120 + N + N^{\text{ctx}}_{\text{val}} + N^{\text{ctx}}_{\text{test}}$.
$\text{StdNorm}(\cdot)$	standardizer	Per-dimension z-score normalization using Stage-0 train-split statistics only.
$\text{vec}(\cdot)$	vectorizer	Row-major flattening of one $H \times d_{\text{exec}}$ action chunk into a length-Hd_{exec} vector.
$\text{NN}_1(\cdot; \mathcal{B})$	nearest neighbor	Euclidean nearest-neighbor query in buffer $\mathcal{B}$ with lexicographic tie breaking.

3.4 Two tiny glossaries

Core method terms. A *VLA* is a vision-language-action model that maps images, language, and robot state to action chunks. A *flow-based* or *flow-matching* action head generates an action chunk by integrating a learned denoising or velocity field rather than by directly emitting a tractable likelihood for the final action. The *execution space* is the robot-side command space after the deterministic adapter Γ_{exec}; the *provider space* is the conditioning format used by the frozen world-model provider after Φ_Ω. A *nuisance model* is a predictor used inside a doubly robust estimator; it need not be perfect, but it should be available before the current real outcome is seen. A *bootstrap head* means one head in a small ensemble trained with independent Bernoulli masks to estimate uncertainty. A *heteroscedastic scale* is an input-dependent predicted standard deviation rather than one global constant.

Software names. *PiPER* is the AgileX single-arm manipulation platform used in the current real study. *GE-Sim* is the released short-horizon predictor in the Genie Envisioner repository. *RLinf* is the public embodied-RL framework used to run the WoVR baseline. *WoVR* is a world-model-heavy VLA post-training method that uses imagined rollouts as the main learning signal. *URDF* stands for *Universal Robot Description Format*, the kinematic model file used for forward kinematics and collision checking. A *FIFO* reservoir is a *first-in-first-out* buffer that evicts the oldest entry when full. *GELU* is the Gaussian Error Linear Unit activation. *AdamW* is Adam with decoupled weight decay.

4 Method

4.1 Step 1: candidate generation from the current $\pi_{0.5}$ backbone

The backbone is the pretrained $\pi_{0.5}$ flow-based VLA from `openpi` [8, 9]. At each decision context, CAVER receives one observation bundle $h_{r,c}$ together with the task language string $g_{r,c}$. The bundle may contain policy-view images, provider-only auxiliary views, robot state, and any camera or sensor metadata required by the concrete instantiation. The method does *not* require those modalities to match across robots; it requires only that the backbone can sample raw candidate chunks from $(h_{r,c}, g_{r,c})$ and that the current execution adapter can deterministically convert them into safety-checkable commands.

Generate $K = 4$ candidate chunks by querying the same round-r backbone with different sampling seeds:

$$\tilde{a}^{\pi}_{r,c,k} = \text{SampleChunk}(\pi_{\theta_r}, h_{r,c}, g_{r,c}; \xi_{r,c,k}), \qquad \xi_{r,c,k} \overset{iid}{\sim} \mathcal{N}(0, I).$$

The raw backbone output $\tilde{a}^{\pi}_{r,c,k}$ is converted once through a deterministic execution adapter

$$a_{r,c,k} = \Gamma_{\text{exec}}\big(\tilde{a}^{\pi}_{r,c,k}, s^{\text{prop}}_{r,c}\big) \in \mathbb{R}^{H \times d_{\text{exec}}}.$$

The only method-level requirements on Γ_{exec} are that it be deterministic, that it produce commands whose semantics are fixed before the study begins, and that it log the exact emitted execution chunk. The present study fixes $H = 4$ and $\Delta t = 0.1$ s, so every candidate covers 0.4 s. In the concrete PiPER study, $d_{\text{exec}} = 7$ and $a_{r,c,k}$ is a joint/gripper chunk defined later in the implementation plan; other robots can replace Γ_{exec} with their own execution semantics while preserving the rest of the algorithm.

Define the normalized candidate-distance metric

$$d_{\text{cand}}(a, a') = \frac{1}{H} \sum_{t=1}^{H} \sqrt{\frac{1}{d_{\text{exec}}} \sum_{i=1}^{d_{\text{exec}}} \left(\frac{a_i[t] - a'_i[t]}{r_i} \right)^2},$$

where $r_i > 0$ are fixed per-dimension normalization ranges declared by the concrete robot instantiation. To enforce diversity, resample candidate k if $\min_{j<k} d_{\text{cand}}(a_{r,c,k}, a_{r,c,j}) < \delta_a$ with the fixed threshold $\delta_a = 0.05$; after at most 16 draws, the latest sample is kept.

4.2 Step 2: frozen off-the-shelf future prediction plus a small utility head

The mainline provider is a frozen off-the-shelf short-horizon predictor $\mathcal{T}_\Omega$. CAVER does *not* assume that the execution-space commands used by the robot are already in the format expected by that provider. It therefore inserts one deterministic compatibility adapter

$$\tilde{a}^{\Omega}_{r,c,k} = \Phi_\Omega(a_{r,c,k}) \in \mathbb{R}^{H \times d_\Omega}.$$

At the method level, Φ_Ω is whatever deterministic map makes the frozen provider consumable. The current study instantiates $\mathcal{T}_\Omega$ with GE-Sim and Φ_Ω with a PiPER-to-GE-Sim compatibility map defined later in the concrete implementation section.

The provider call is written abstractly as

$$\hat{I}_{r,c,k,1:H} = \mathcal{T}_\Omega(h_{r,c}, \tilde{a}^{\Omega}_{r,c,k}).$$

The bundle $h_{r,c}$ contributes whatever views and calibration metadata the provider requires. In the current study, GE-Sim consumes multiview RGB together with camera intrinsics/extrinsics and the concrete provider chunk $\tilde{a}^{\text{GE}}_{r,c,k} = \Phi_{\text{GE}}(a_{r,c,k})$.

The main method does *not* train a new world model. It freezes $\mathcal{T}_\Omega$ and extracts a visual feature from one fixed provider summary output selected by the concrete instantiation. Let F^{frz}_ρ be the frozen visual tower used for this purpose. In the current study, the summary output is the final predicted `head` view frame, so

$$q_{r,c,k} = R\, F^{\text{frz}}_\rho(\hat{I}^{\text{head}}_{r,c,k,H}) \in \mathbb{R}^{64}.$$

Other robots/providers can change only this summary readout choice without changing the selector, DR correction, or backbone update rule.

A small utility head produces a cheap scalar estimate

$$\hat{v}^{(b)}_{r,c,k} = V^{(b)}_\omega(e_{r,c}, q_{r,c,k}, \phi_a(a_{r,c,k})), \qquad b = 1, 2, 3,$$

where $e_{r,c} = E^{\text{frz}}_\phi(h_{r,c}, g_{r,c})$ is the frozen context embedding from the same $\pi_{0.5}$ backbone and ϕ_a is the normalized flattened action encoding defined in Step 3. The shared trunk is a 2-layer MLP with hidden width 256 and GELU activations; the last layer branches into three scalar heads. The heads are trained with independent Bernoulli bootstrap masks

$$m^{(b)}_{c,k} \overset{iid}{\sim} \text{Bernoulli}(0.8),$$

resampled once per epoch. The final proxy value and pre-calibration uncertainty are

$$\hat{v}_{r,c,k} = \frac{1}{3} \sum_{b=1}^{3} \hat{v}^{(b)}_{r,c,k}, \qquad \hat{\sigma}^{\text{pre}}_{r,c,k} = \sqrt{\frac{1}{3} \sum_{b=1}^{3} \left(\hat{v}^{(b)}_{r,c,k} - \hat{v}_{r,c,k} \right)^2}.$$

The utility head is trained only in Stage 0 and on the shared seed real dataset, then frozen for the entire matched-budget real study. This is the only trainable component sitting directly on top of the off-the-shelf provider, and it is deliberately lightweight so the paper remains about verification rather than world-model training.

Explicit proxy target. The utility head is trained on

$$u_{c,k}^{\text{proxy}} = 0.5\, s_{c,k}^{\text{bin}} + 0.5\, q_{c,k}^{\text{prog}},$$

where $s_{c,k}^{\text{bin}} \in \{0,1\}$ is the binary episode-success label obtained by executing candidate k to episode termination in Stage 0 or in the shared seed real set, and $q_{c,k}^{\text{prog}} \in [0,1]$ is a normalized progress score. Table 2 defines the progress-score formulas used for Stage 0 and seed-data labeling.

Table 2: Exact progress-score formulas used for Stage 0 and seed-data labeling.

Task	Progress score $q_{c,k}^{\text{prog}}$
Block-to-tray / can-to-bowl	If d^{start} and d^{end} are the object-center distances to the receptacle center before and after the chunk, use $\text{clip}((d^{\text{start}} - d^{\text{end}})/(d^{\text{start}} + 10^{-3}), 0, 1)$.
Two-block stack	Let d_{xy} be horizontal distance between upper and lower block centers and h the upper-block height. Use $0.5\,\text{clip}((d_{xy}^{\text{start}} - d_{xy}^{\text{end}})/(d_{xy}^{\text{start}} + 10^{-3}), 0, 1) + 0.5\,\text{clip}((h^{\text{end}} - h^{\text{start}})/(h^{\star} - h^{\text{start}} + 10^{-3}), 0, 1)$, where $h^{\star}$ is the target supported stacking height.
Relocate to marked region	If d_{region} is the planar distance from the object center to the target-region center, use $\text{clip}((d_{\text{region}}^{\text{start}} - d_{\text{region}}^{\text{end}})/(d_{\text{region}}^{\text{start}} + 10^{-3}), 0, 1)$.
Drawer / handle open	If o^{start} and o^{end} are the drawer-joint opening fractions and $o^{\max} = 1$, use $\text{clip}((o^{\text{end}} - o^{\text{start}})/(o^{\max} - o^{\text{start}} + 10^{-3}), 0, 1)$.

Let $\mathcal{I}_V$ denote the union of all Stage-0 executed candidate tuples and all executed tuples in the shared seed real set. The utility-head loss is

$$L_V(\omega) = \sum_{b=1}^{3} \sum_{(c,k)\in\mathcal{I}_V} m_{c,k}^{(b)} \Big(V_\omega^{(b)}(e_c, q_{c,k}, \phi_a(a_{c,k})) - u_{c,k}^{\text{proxy}} \Big)^2.$$

In the mainline method, no future-prediction loss is optimized inside CAVER because the future predictor itself is frozen off-the-shelf.

4.3 Step 3: exact feature construction with non-circular novelty

The base feature is defined before novelty is computed, so there is no circular dependence.

First define the action summary

$$\phi_a(a_{r,c,k}) = \text{vec}(\text{StdNorm}(a_{r,c,k})) \in \mathbb{R}^{H d_{\text{exec}}},$$

that is, flatten the $H \times d_{\text{exec}}$ execution chunk after per-dimension normalization using Stage-0 training statistics. Next define the within-context disagreement

$$D_{r,c,k} = \frac{1}{K-1} \sum_{j\neq k} \frac{1}{H} \|a_{r,c,k} - a_{r,c,j}\|_2.$$

The bootstrap uncertainty from the utility head is

$$\hat{\sigma}_{r,c,k}^{\text{pre}} = \sqrt{\frac{1}{B} \sum_{b=1}^{B} (\hat{v}_{r,c,k}^{(b)} - \hat{v}_{r,c,k})^2}, \qquad B = 3.$$

Now define the *base feature vector*

$$b_{r,c,k} = \left[\hat{v}_{r,c,k},\ q_{r,c,k},\ \phi_a(a_{r,c,k}),\ D_{r,c,k},\ \hat{\sigma}^{\text{pre}}_{r,c,k}\right].$$

Only after this base vector is built do we compute novelty:

$$N_{r,c,k} = \begin{cases} 0, & |\mathcal{B}_{\text{hist}}| < 32, \\ \frac{1}{d_b} \left\| b_{r,c,k} - \text{NN}_1(b_{r,c,k}; \mathcal{B}_{\text{hist}}) \right\|_2, & \text{otherwise}, \end{cases}$$

where $\mathcal{B}_{\text{hist}}$ is a FIFO reservoir containing the most recent 1024 executed base-feature vectors and d_b is the dimension of $b_{r,c,k}$. The final feature vector is

$$z_{r,c,k} = [b_{r,c,k}, N_{r,c,k}].$$

This removes the recursion completely.

4.4 Step 4: frozen-form stochastic selector with refreshed calibrated inputs

Before round 1 begins, CAVER fits one *initial* calibrator f_{ψ_0} on a fully verified warm-start set $\mathcal{D}_{\text{seed}}$ using the same heteroscedastic loss as later rounds. In Stage 1, $\mathcal{D}_{\text{seed}}$ is the shared 120-context PiPER seed set. In Stage 0 simulation sweeps, $\mathcal{D}_{\text{seed}}$ is a matched simulated seed set of 120 fully verified contexts, balanced as 24 contexts per Stage-0 task. Round 1 therefore *does* start with a learned calibrator; there is no raw-proxy-only special case in the main protocol.

At the start of round r, use the lagged calibrator from the previous round:

$$\bar{v}_{r,c,k} = \mu_{r-1}(z_{r,c,k}), \qquad \bar{\sigma}_{r,c,k} = \sigma_{r-1}(z_{r,c,k}), \qquad r \geq 1,$$

with the convention that (μ_0, σ_0) are the outputs of f_{ψ_0} fit on $\mathcal{D}_{\text{seed}}$. Then score candidate k by

$$\text{ranknorm}(\bar{v}_{r,c,k}) = \frac{1}{K-1} \sum_{j \neq k} \mathbb{I}\{\bar{v}_{r,c,j} \leq \bar{v}_{r,c,k}\},$$

$$s_{r,c,k} = \lambda_V \,\text{ranknorm}(\bar{v}_{r,c,k}) + \lambda_U \,\bar{\sigma}_{r,c,k} + \lambda_D \, D_{r,c,k} + \lambda_N \, N_{r,c,k}.$$

The scalar coefficients $\lambda_V, \lambda_U, \lambda_D, \lambda_N$, the selector temperature T, the exploration floor ϵ, the conservative multiplier κ, and the acceptance threshold τ are tuned once on Stage-0 validation and then frozen for the entire real study.

Turn the scores into a raw stochastic selector

$$\rho^{\text{raw}}_r(k \mid c) = \frac{\exp(s_{r,c,k}/T)}{\sum_{j=1}^{K} \exp(s_{r,c,j}/T)}, \qquad \nu^{\text{raw}}_r(k \mid c) = (1-\epsilon)\rho^{\text{raw}}_r(k \mid c) + \epsilon \frac{1}{K}.$$

After the deterministic safety mask is applied, the *actual* execution selector is

$$\nu_r(k \mid c) = \frac{\nu^{\text{raw}}_r(k \mid c)\, \mathbb{I}\{k \in \mathcal{K}^{\text{safe}}_{r,c}\}}{\sum_{j=1}^{K} \nu^{\text{raw}}_r(j \mid c)\, \mathbb{I}\{j \in \mathcal{K}^{\text{safe}}_{r,c}\}}, \qquad p_{r,c,k} = \nu_r(k \mid c),$$

defined whenever $\mathcal{K}^{\text{safe}}_{r,c} \neq \varnothing$. Then sample

$$A_{r,c} \sim \nu_r(\cdot \mid c).$$

Because ν^{raw}_r has an exploration floor and the safety mask is deterministic, every safety-approved candidate has strictly positive exact propensity and the logged propensity is exact on the feasible set $\mathcal{K}^{\text{safe}}_{r,c}$.

4.5 Step 4b: deterministic safety shield before any real execution

All real methods share one identical hard safety shield S_{safe}. The shield is *not* learned, does *not* depend on the proxy value, and is applied after candidate chunks have been formed but before the selector can execute anything on hardware. At the method level, CAVER requires only that

$$S_{\text{safe}}(a_{r,c,k}, s^{\text{prop}}_{r,c}) \in \{0,1\}$$

be a deterministic auditable test of hard robot constraints. In the current study, these constraints are joint/gripper bounds, velocity and acceleration caps, workspace limits, self-collision, table collision, and runtime abort rules; the literal PiPER constants are pinned in the concrete implementation section and appendix.

The safe candidate set is

$$\mathcal{K}^{\text{safe}}_{r,c} = \{k \in \{1,\dots,K\} : S_{\text{safe}}(a_{r,c,k}, s^{\text{prop}}_{r,c}) = 1\}.$$

Unsafe candidates are masked by setting their selector logits to $-\infty$ and renormalizing the selector on $\mathcal{K}^{\text{safe}}_{r,c}$. If $\mathcal{K}^{\text{safe}}_{r,c} = \varnothing$, the bridge sends a 0.4 s hold command in the current execution space, records a `safety_abort` flag, terminates the episode immediately, and counts that decision context against the matched training budget N because it consumed a real decision opportunity. Safety-abort contexts are excluded from the DR tuple set and from all update buffers.

Execution is also monitored online by the same shield. In the present PiPER study, that monitor runs at 20 Hz and enforces the same hard workspace/collision rules plus heartbeat and tracking-error aborts. These rules are simple engineering guardrails rather than the scientific contribution of the paper, but they are necessary for a credible hardware study and are consistent with the spirit of recent VLA safety work [18, 19].

4.6 Step 5: real execution and verified labels

For contexts with $\mathcal{K}^{\text{safe}}_{r,c} \neq \varnothing$, execute the chosen chunk $a_{r,c,A_{r,c}}$ on the current robot bridge at the locked control rate Δt^{-1}, then continue the episode under the same round-r backbone policy until success or timeout. The primary verified label is the binary episode-success outcome

$$Y_{r,c} = Y_{r,c}(A_{r,c}) \in \{0,1\}.$$

Safety-abort contexts with $\mathcal{K}^{\text{safe}}_{r,c} = \varnothing$ do not define $Y_{r,c}$ for a policy candidate and therefore never enter the DR tuple set or the backbone-update buffers.

At the method level, CAVER requires one deterministic verified labeler

$$M_{\text{task}}(\text{executed episode trace}) \in \{0,1\}$$

shared across methods, optionally together with a diagnostic continuous progress score $\bar{Y}_{r,c} \in [0,1]$. In the current study, M_{task} is a marker-based geometric end-state checker for three PiPER tasks with a fixed human-audit path for low-confidence marker tracks; the exact thresholds, audit triggers, and geometric rules are pinned in the concrete implementation section and appendix. The important method-level point is that the labeler is deterministic, auditable, and shared across CAVER, real-only π-StepNFT, and WoVR when they are evaluated on real execution.

4.7 Step 6: lagged doubly robust correction

The online implementation uses a lagged nuisance model to avoid same-round leakage. Before round r begins, define

$$m_{r-1}(z) = \mu_{r-1}(z), \qquad r \geq 1,$$

where μ_0 is the initial calibrator mean fit on the warm-start seed set $\mathcal{D}_{\text{seed}}$. Thus round 1 already uses the seed-trained calibrator, and every later round uses the previous round's refit calibrated mean. For every *safety-approved* candidate $k \in \mathcal{K}^{\text{safe}}_{r,c}$ in the current round, define the doubly robust pseudo-outcome

$$\tilde{y}^{\text{DR}}_{r,c,k} = m_{r-1}(z_{r,c,k}) + \frac{\mathbb{I}\{A_{r,c} = k\}}{p_{r,c,k}}\big(Y_{r,c} - m_{r-1}(z_{r,c,k})\big).$$

Because m_{r-1} is measurable with respect to the history available before round r, this is the sequential version of the usual doubly robust correction and keeps the online algorithm single-pass. For retrospective diagnostics on the accumulated dataset, the codebase will also report a 5-fold cross-fitted estimate whose nuisance family is exactly the same width-256 2-layer GELU MLP used for the online calibrator; each fold trains on 80% of the tuples and predicts the held-out 20%.

Fit a calibrator after collecting round-r data:

$$(\mu_r(z), \log \sigma_r^2(z)) = f_{\psi_r}(z),$$

with loss

$$L_{\mathrm{cal}}(\psi_r) = \sum_{(r',c,k)\in\mathcal{D}^{\mathrm{tuple}}_{\le r}} \left[\frac{(\tilde{y}^{\mathrm{DR}}_{r',c,k} - \mu_r(z_{r',c,k}))^2}{2\sigma_r^2(z_{r',c,k})} + \frac{1}{2}\log \sigma_r^2(z_{r',c,k}) \right].$$

Here

$$\mathcal{D}^{\mathrm{tuple}}_{\le r} = \left\{ (r', c, k, z_{r',c,k}, \tilde{y}^{\mathrm{DR}}_{r',c,k}) : 1 \le r' \le r,\ k \in \mathcal{K}^{\mathrm{safe}}_{r',c} \right\}$$

denotes the accumulated *candidate-tuple* dataset from all verified contexts collected up to and including round r; safety-abort contexts are omitted because no policy candidate was executed. The calibrator is a 2-layer MLP with hidden width 256, GELU activations, and output clipping $\sigma_r(z) \in [0.02, 0.5]$ for numerical stability. Because $\tilde{y}^{\mathrm{DR}}$ can fall outside $[0, 1]$, the mean head $\mu_r(z)$ is *not* clipped inside the training loss.

For probability-style diagnostics, define a clipped score

$$\bar{\mu}_r(z) = \mathrm{clip}(\mu_r(z), 0, 1).$$

Whenever the proposal reports Brier score or ECE for CAVER's *online* calibration signal, it uses $\bar{\mu}_r(z_{r,c,A_{r,c}})$ on the executed candidate only. Cross-method Brier/ECE comparisons use the common post-hoc diagnostic described in the experimental section so that real-only and WoVR are also well defined.

4.8 Step 7: confident executed updates only

Define the conservative utility

$$\mathrm{LCB}_r(z) = \mu_r(z) - \kappa\sigma_r(z).$$

Every executed trajectory is placed into exactly one cumulative buffer:

$$\mathcal{M}^+_{\le r} = \mathcal{M}^+_{<r} \cup \left\{ \tau_{r,c} : Y_{r,c} = 1 \text{ and } \mathrm{LCB}_r(z_{r,c,A_{r,c}}) > \tau \right\},$$

$$\mathcal{M}^-_{\le r} = \mathcal{M}^-_{<r} \cup \left\{ \tau_{r,c} : Y_{r,c} = 0 \text{ or } \mathrm{LCB}_r(z_{r,c,A_{r,c}}) \le 0 \right\},$$

$$\mathcal{M}^0_{\le r} = \mathcal{M}^0_{<r} \cup \left\{ \tau_{r,c} : Y_{r,c} = 1 \text{ and } 0 < \mathrm{LCB}_r(z_{r,c,A_{r,c}}) \le \tau \right\}.$$

Thus successful samples with $0 < \mathrm{LCB}_r \le \tau$ are assigned to an explicit *abstain* region. They are kept for calibrator fitting but are never used as positives or negatives in the backbone update.

Negative pairing rule. For every positive trajectory $\tau^+ \in \mathcal{M}^+_{\le r}$, choose one negative trajectory $\tau^- \in \mathcal{M}^-_{\le r}$ from the same task whose start-state embedding is nearest in cosine distance to the positive start-state embedding. Start-state embeddings are extracted from the frozen context encoder E^{frz}_ϕ before any LoRA adaptation is applied. If no same-task negative exists, use the nearest negative from any task but record that fallback event in the logs. Negative reuse is *allowed*: one negative trajectory may be paired with multiple positives within the same update block or across rounds. Under fixed random seeds, pair construction is deterministic: positives are processed in ascending global trajectory ID, cosine-distance ties are broken by ascending negative trajectory ID, and the same cumulative buffers therefore yield the same pair set $\mathcal{P}_{\le r}$. The real-only baseline uses the same pair-construction rule on its admitted executed trajectories.

Backbone update with the explicit π-StepNFT objective. Let a matched positive–negative pair be denoted (τ^+, τ^-). This proposal keeps the *mirrored-branch ranking form* of π-StepNFT but pins the previously implicit backend details in-text so the study is reproducible without consulting external code.

Pinned solver transition sampler. Every executed chunk stores the 51 solver states $\{x_0, \ldots, x_{50}\}$ produced by the local `pi05_piper_h4` flow solver over normalized solver time $t \in \{0, \ldots, 49\}$ with $\Delta_s = 1/50$ and one-step successor notation $x_{t^-} := x_{t+1}$. For every optimizer epoch and every pair (τ^+, τ^-), define $\mathcal{S}(\tau^+, \tau^-)$ by sampling exactly 8 transition tuples with replacement: 4 tuples from τ^+ and 4 tuples from τ^-. Within each chosen trajectory, sample the solver index uniformly,

$$t \sim \mathrm{Unif}\{0, 1, \ldots, 49\},$$

and record (x_t, x_{t^-}, c, t) together with the trajectory label

$$y = \begin{cases} +1, & \text{if the sampled tuple came from } \tau^+, \\ -1, & \text{if the sampled tuple came from } \tau^-. \end{cases}$$

Pinned mirrored branches and one-step moments. Let

$$v_\theta^{\mathrm{old}} = \pi_{\theta_{\mathrm{roll}}}(c, x_t, t), \qquad v_\theta = \pi_\theta(c, x_t, t), \qquad \Delta v_\theta = v_\theta - v_\theta^{\mathrm{old}},$$

where θ_{roll} is the frozen rollout policy that collected the executed data. The mirrored StepNFT branches are fixed at

$$v_\theta^+ = (1 - \beta)\, v_\theta^{\mathrm{old}} + \beta\, v_\theta, \qquad v_\theta^- = (1 + \beta)\, v_\theta^{\mathrm{old}} - \beta\, v_\theta, \qquad \beta = 0.05.$$

In this study, the one-step moments are the explicit Euler–Maruyama moments

$$\mu_{\theta,t}^{\pm} = x_t + \Delta_s\, v_\theta^{\pm}(c, x_t, t), \qquad \Sigma_t = 2\Delta_s I.$$

Define the variance-normalized step errors

$$E_{\theta,t}^{\pm} = \left\| x_{t^-} - \mu_{\theta,t}^{\pm} \right\|_{\Sigma_t^{-1}}^2.$$

Pinned per-step and backbone loss. With trust-region penalty weight $\lambda_{\mathrm{TR}} = 10^{-4}$, the per-step loss is

$$\ell_t(\theta) = \mathrm{softplus}\left(\frac{1}{2}\, y\, (E_{\theta,t}^+ - E_{\theta,t}^-) \right) + \lambda_{\mathrm{TR}} \|\Delta v_\theta\|_2^2.$$

If $\mathcal{P}_{\le r}$ is the set of cumulative positive–negative pairs available after round r, the backbone objective is

$$L_{\mathrm{PT}}(\theta) = \frac{1}{|\mathcal{P}_{\le r}|} \sum_{(\tau^+, \tau^-) \in \mathcal{P}_{\le r}} \frac{1}{|\mathcal{S}(\tau^+, \tau^-)|} \sum_{(x_t, x_{t^-}, c, t) \in \mathcal{S}(\tau^+, \tau^-)} \ell_t(\theta).$$

Pinned optimizer and trainable modules. The backbone optimizer is AdamW with learning rate 10^{-5}, betas $(0.9, 0.999)$, weight decay 10^{-4}, gradient clip 1.0, batch size 16 positive–negative pairs, and exactly 500 gradient steps whenever an update is triggered. The common seed warm-start pass also uses exactly 500 gradient steps with the same optimizer and no early stopping. LoRA is inserted *only* into the action-head denoising transformer blocks: the trainable linear projections are $\{q\_proj, k\_proj, v\_proj, out\_proj, fc1, fc2\}$ in every action-head block, with LoRA rank 16, scale $\alpha = 32$, and dropout 0. No LoRA modules are attached to the vision tower, language tower, frozen context encoder, provider, proxy head, or calibrator.

The mainline backend therefore keeps the StepNFT-style mirrored-branch ranking loss and identical optimizer settings for CAVER and the real-only baseline. CAVER does not alter the backbone objective in the headline comparison. The only mainline difference between CAVER and the real-only baseline is *which executed examples reach this unchanged trainer*. A conservative weighted variant is evaluated separately as an ablation.

When an update is skipped. If $|\mathcal{M}_{\le r}^+| < 8$, the code performs *no* backbone update after round r. This is the concrete meaning of "accumulate more data": the current round still enlarges $\mathcal{M}_{\le r}^+$, $\mathcal{M}_{\le r}^-$, and $\mathcal{M}_{\le r}^0$, but the LoRA optimizer is not called until at least eight cumulative positives exist. Only LoRA modules inside the action head are updated by L_{PT}. The encoders used to form $z_{r,c,k}$ stay frozen.

4.9 Algorithm summary

Algorithm 1 CAVER

Initialize the common seed-warmed backbone π_{θ_1} from the shared seed set $\mathcal{D}_{\text{seed}}$, frozen provider $\mathcal{T}_\Omega$, deterministic adapters $\Gamma_{\text{exec}}, \Phi_\Omega$, frozen future encoder F_ρ^{frz}, and Stage-0-trained proxy head V_ω.
Fit the initial calibrator f_{ψ_0} on $\mathcal{D}_{\text{seed}}$ before any round-1 context is collected.
Tune selector coefficients $(\lambda_V, \lambda_U, \lambda_D, \lambda_N, T, \epsilon, \kappa, \tau)$ on Stage 0 only and freeze them.
Initialize cumulative buffers $\mathcal{M}^+_{\le 0} = \mathcal{M}^-_{\le 0} = \mathcal{M}^0_{\le 0} = \varnothing$.
for round $r = 1, 2, \ldots, R$ **do**
    for each decision context c collected in round r **do**
        Sample K raw candidate chunks from the current round-r backbone π_{θ_r}.
        Convert each chunk with Γ_{exec} for execution and Φ_Ω for provider conditioning.
        Run the frozen provider on each candidate and extract the summary future embedding $q_{r,c,k}$.
        Compute cheap features $z_{r,c,k}$ and selector scores $s_{r,c,k}$.
        Apply the deterministic safety shield, form $\mathcal{K}^{\text{safe}}_{r,c}$, and mask unsafe candidates.
        if $\mathcal{K}^{\text{safe}}_{r,c} = \varnothing$ **then**
            Send the 0.4 s hold command in the current execution space, log `safety_abort`, count the context against N, and terminate the episode.
            continue
        end if
        Sample one candidate $A_{r,c} \sim \nu_r(\cdot \mid c)$ on the safe subset and log all propensities $p_{r,c,1:K}$.
        Execute $a_{r,c,A_{r,c}}$ on the robot, continue the episode with π_{θ_r}, and log $Y_{r,c} = M_{\text{task}}$(executed episode trace).
        Append executed base feature $b_{r,c,A_{r,c}}$ to $\mathcal{B}_{\text{hist}}$.
    end for
    Form lagged DR pseudo-outcomes $\tilde{y}^{\text{DR}}_{r,c,k}$ for the newly collected non-aborted contexts.
    Refit calibrator $(\mu_r, \sigma_r) = f_{\psi_r}(z)$ on all accumulated candidate tuples $\mathcal{D}^{\text{tuple}}_{\le r}$.
    Add each executed trajectory to exactly one of $\mathcal{M}^+_{\le r}, \mathcal{M}^-_{\le r}, \mathcal{M}^0_{\le r}$ using the LCB rules.
    if $|\mathcal{M}^+_{\le r}| \ge 8$ **then**
        Build cumulative positive–negative pairs $\mathcal{P}_{\le r}$ and update only action-head LoRA parameters with L_{PT}.
        Set $\pi_{\theta_{r+1}}$ to the updated backbone.
    else
        Skip the backbone update and keep $\pi_{\theta_{r+1}} = \pi_{\theta_r}$.
    end if
end for

5 Theory: why the selective-verification layer is sensible

The theory is intentionally about the verification layer, not about the entire deep robot pipeline. It justifies why exact logged propensities and a small set of real outcomes can correct a much larger cheap pool. The deployed code does *not* solve an online selector-class optimization problem over a large Π each round. Instead, it freezes one parametric selector family after Stage 0 and uses the theorem as an evaluative idealization showing why exact propensities and conservative confidence accounting are worthwhile.

5.1 Assumptions

Assumption A1 (Roundwise frozen data-collection policy)**.** Within a given round r, the continuation policy π_{θ_r}, the selector ν_r, and the lagged nuisance model m_{r-1} are fixed before any round-r contexts are collected.

Assumption A2 (Exact randomized verification on the feasible set)**.** At every round and context with $\mathcal{K}^{\text{safe}}_{r,c} \neq \varnothing$, one safety-approved candidate is selected with logged probability $p_{r,c,k} = \nu_r(k \mid c)$ for every $k \in \mathcal{K}^{\text{safe}}_{r,c}$, and $p_{r,c,k} > 0$ on that feasible set.

Assumption A3 (Bounded binary utility on feasible candidates)**.** For every round, context, and safety-approved candidate $k \in \mathcal{K}^{\text{safe}}_{r,c}$, the potential outcome satisfies $Y_{r,c}(k) \in \{0, 1\}$.

5.2 Conditional unbiasedness of the roundwise DR estimator

Let $\mathcal{H}_{r-1}$ denote the full history available before round r begins. For any fixed target selector $\nu(\cdot \mid c)$ over the round-r feasible candidate set, define its roundwise utility

$$U_r(\nu) = \mathbb{E}\left[\sum_{k \in \mathcal{K}^{\text{safe}}_{r,c}} \nu(k \mid c) Y_{r,c}(k) \,\middle|\, \mathcal{H}_{r-1}\right].$$

Define the roundwise DR estimator

$$\hat{U}^{\text{DR}}_r(\nu) = \frac{1}{N_r} \sum_{c=1}^{N_r} \left[\sum_{k \in \mathcal{K}^{\text{safe}}_{r,c}} \nu(k \mid c) m_{r-1}(z_{r,c,k}) + \frac{\nu(A_{r,c} \mid c)}{p_{r,c,A_{r,c}}} \big(Y_{r,c} - m_{r-1}(z_{r,c,A_{r,c}})\big)\right],$$

where N_r is the number of verified, non-aborted contexts collected in round r.

Theorem 5.1 (Conditional unbiasedness)**.** *Under Assumptions A1–A3,*

$$\mathbb{E}\big[\hat{U}^{\text{DR}}_r(\nu) \mid \mathcal{H}_{r-1}\big] = U_r(\nu)$$

for any fixed target selector ν.

The theorem is the right level of guarantee for this paper. It says that even though CAVER executes only one candidate per context, the value of any selector on the deterministic feasible set can still be estimated without bias, conditional on the past, as long as propensities are exact.

5.3 High-confidence improvement over a baseline selector

Let Π be a finite class of candidate selectors containing a baseline ν_b. Suppose that for every $\nu \in \Pi$ there exists a confidence radius $\beta_r(\nu, \delta)$ such that with probability at least $1 - \delta$,

$$|\hat{U}^{\text{DR}}_r(\nu) - U_r(\nu)| \le \beta_r(\nu, \delta)$$

simultaneously for all $\nu \in \Pi$. Define

$$\text{LCB}_r(\nu) = \hat{U}^{\text{DR}}_r(\nu) - \beta_r(\nu, \delta), \qquad \text{UCB}_r(\nu) = \hat{U}^{\text{DR}}_r(\nu) + \beta_r(\nu, \delta).$$

Choose

$$\hat{\nu}_r = \operatorname*{argmax}_{\nu \in \Pi} \text{LCB}_r(\nu) \quad \text{subject to} \quad \text{LCB}_r(\nu) \ge \text{UCB}_r(\nu_b).$$

If no selector satisfies the constraint, deploy ν_b.

Theorem 5.2 (Roundwise high-confidence improvement)**.** *With probability at least $1 - \delta$,*

$$U_r(\hat{\nu}_r) \ge U_r(\nu_b).$$

Corollary 5.3 (Why support and budget matter)**.** *If β_r is instantiated with an empirical Bernstein bound or with a self-normalized importance-weighted bound, then the ability to certify improvement improves as the verified budget N_r grows and worsens when the feasible set is large or the exploration floor is too small. Informally,*

$$N_r = \tilde{\mathcal{O}}\left(\frac{\max_c |\mathcal{K}^{\text{safe}}_{r,c}|}{\epsilon\, \Delta^2} \log \frac{|\Pi|}{\delta}\right)$$

verified contexts are sufficient to certify an improvement margin Δ.

Remark. This theory is a *conditional context-level* statement. It does *not* prove monotonic improvement of the full VLA policy after LoRA updates, and it does *not* control how earlier action choices change which future contexts are encountered. It proves something narrower and still useful: conditional on the realized stream of decision contexts, the real-verification decision layer can be evaluated and improved with principled uncertainty accounting even when only one candidate per context is actually executed.

6 Experimental plan

6.1 Research questions

RQ1. Can a small number of verified real executions correct a much larger cheap pool of provider-based signals?

RQ2. Under the same verified real budget, does CAVER outperform real-only π-StepNFT on the same $\pi_{0.5}$ backbone?

RQ3. Under the same verified real budget, does CAVER transfer better to the real robot than world-model-heavy post-training that trusts imagined rollouts more directly?

6.2 Stage 0: five-task LIBERO stage

Stage 0 is the full debugging and hyperparameter-tuning stage. It uses five LIBERO tasks:

- pick block to tray,
- place can into bowl,
- stack two blocks,
- relocate object to a marked goal region,
- open a simple drawer or cabinet handle.

All methods use the same decision protocol: $K = 4$ candidates, $H = 4$ executed commands per chunk, and verified budgets $N \in \{25, 50, 100, 200\}$ contexts per task. Every Stage-0 experiment uses three random seeds.

Exact selector-tuning rule. Hyperparameters $(\lambda_V, \lambda_U, \lambda_D, \lambda_N, T, \epsilon, \kappa, \tau)$ are tuned *once* on Stage-0 validation by the scalar model-selection score

$$J_{\mathrm{S0}} = \overline{\mathrm{Succ}}_{\mathrm{val}}(N = 100) - 0.25\,\overline{\mathrm{ECE}}_{\mathrm{val}} - 0.10\,\overline{\mathrm{Brier}}_{\mathrm{val}},$$

where the overline denotes the average across the five Stage-0 tasks and three random seeds. J_{S0} is *not* a training loss; it is only the rule used to freeze one selector setting for the rest of the study. If multiple settings are within one bootstrap standard error of the best J_{S0}, choose the rightmost tie-break default from Table 3.

Stage-0 simulated warm-start set. Every Stage-0 run begins with one fixed simulated seed set of 120 fully verified contexts, balanced as 24 contexts per Stage-0 task. This set is used in exactly the same way that the real seed set will later be used in Stage 1: first produce a common seed-warmed backbone checkpoint by one uniform-selector StepNFT warm-start pass of exactly 500 gradient steps, then fit the initial calibrator f_{ψ_0} on the same fully verified tuples, and only then start counting the Stage-0 training budget N.

Strict Stage-0 no-leakage policy. Before any Stage-0 sweep begins, fix four *disjoint* template lists for each LIBERO task:

$$\mathcal{T}_{\mathrm{S0}}^{\mathrm{seed}}, \qquad \mathcal{T}_{\mathrm{S0}}^{\mathrm{train}}, \qquad \mathcal{T}_{\mathrm{S0}}^{\mathrm{val}}, \qquad \mathcal{T}_{\mathrm{S0}}^{\mathrm{test}}.$$

The simulated seed list $\mathcal{T}_{\mathrm{S0}}^{\mathrm{seed}}$ contains the 24 warm-start contexts per task mentioned above. The validation list $\mathcal{T}_{\mathrm{S0}}^{\mathrm{val}}$ contains 20 held-out episodes per task and is used *only* for the selector-selection score J_{S0} and checkpoint selection in Stage 0. The audit list $\mathcal{T}_{\mathrm{S0}}^{\mathrm{test}}$ contains 20 disjoint held-out episodes per task and is opened once only for final Stage-0 sanity plots. All budgeted Stage-0 online training contexts counted in N are collected exclusively from $\mathcal{T}_{\mathrm{S0}}^{\mathrm{train}}$. The proxy head V_ω may be fit only on executed tuples from $\mathcal{T}_{\mathrm{S0}}^{\mathrm{train}}$ together with the shared Stage-1 real seed tuples; it may never use $\mathcal{T}_{\mathrm{S0}}^{\mathrm{val}}$ or $\mathcal{T}_{\mathrm{S0}}^{\mathrm{test}}$. The calibrator in Stage-0 dry runs may use only the simulated seed tuples plus accumulated training tuples from $\mathcal{T}_{\mathrm{S0}}^{\mathrm{train}}$. Because GE-Sim is frozen in the mainline method, there is no Stage-0 provider-training split. The membership of every template ID in these four partitions is written to `manifest.lock` before the first Stage-0 sweep and is never changed mid-study.

Shared seed real set. Before the matched-budget Stage-1 study, collect one common warm-start dataset of **120** PiPER decision contexts total, balanced as 40 contexts for each of the three physical Stage-1 tasks (block-to-tray, can-to-bowl, and stack). The seed set is collected with the initial $\pi_{0.5}$ backbone and a uniform random selector. Its allowed use is locked:

- **All methods:** run one identical seed warm-start pass of the unchanged StepNFT backend on the executed seed trajectories, using exactly 500 gradient steps with the pinned optimizer from Step 7, producing the common starting checkpoint π_{θ_1} used by CAVER, real-only π-StepNFT, WoVR, and the seed-only anchor baseline.
- **CAVER only:** additionally fit the frozen proxy head V_ω and the initial calibrator f_{ψ_0} on the same seed tuples.
- **WoVR only:** the same seed tuples may also initialize any observation normalizers or reward-scale statistics required by the public RLinf implementation, but may not be supplemented by extra real data.

No method may collect extra real interactions before the matched-budget curve starts. The shared seed set is therefore a *common warm-start cost*, reported separately from the matched-budget training curves rather than silently folded into them.

6.3 Stage 1: three-task PiPER study

The real study uses exactly three tasks: block-to-tray, can-to-bowl, and two-block stack. Each Stage-1 training run is task-specific, each episode permits at most four decision contexts, and each round contains exactly $B_{\text{round}} = 25$ counted training contexts. Therefore the main *training* budget curve $N \in \{25, 50, 100, 200\}$ corresponds to exactly $R = N/25 \in \{1, 2, 4, 8\}$ rounds per task and per method, together with one mandatory $N = 0$ seed-only anchor point that corresponds to the common seed-warmed checkpoint π_{θ_1} before any Stage-1 training. For the three main trainable methods (CAVER, real-only π-StepNFT, and WoVR), the study records two independent full training runs at every budget and a third independent run at $N = 100$ as the primary stability point.

Budget accounting rule. Only on-policy executed decision contexts collected during Stage-1 training count toward N. The shared 120-context seed set does not count toward N. Validation and test episodes also do *not* count toward N and are reported separately under "evaluation interactions." Every results table and success curve therefore reports two x-axes: the matched *training budget* N and the corresponding *total real interaction count*

$$N_{\text{total}} = 120 + N + N_{\text{val}}^{\text{ctx}} + N_{\text{test}}^{\text{ctx}},$$

where 120 is the common seed warm-start cost and $N_{\text{val}}^{\text{ctx}}, N_{\text{test}}^{\text{ctx}}$ are the realized numbers of executed decision contexts consumed by validation and test episodes. The mandatory seed-size sweep repeats the same accounting for each $N_{\text{seed}} \in \{0, 40, 80, 120\}$ so readers can distinguish warm-start dependence from true Stage-1 sample efficiency.

Held-out evaluation. Every compared checkpoint is selected by the same rule: best mean success on 10 validation episodes per task drawn from a fixed held-out template list. Final reported success is mean success on 30 held-out real episodes per task drawn from a second disjoint template list shared across methods and runs, with a nonparametric bootstrap confidence interval over episodes.

Repeated-run and paired statistical plan. The final paper will report both across-run and within-run uncertainty. For each method, task, and budget, aggregate the two independent training runs (three at $N = 100$) with a hierarchical bootstrap that first resamples runs and then resamples episodes within each run. In addition, the headline $N = 100$ comparison uses a paired permutation test over the 30 shared held-out test templates for CAVER versus the real-only baseline and for CAVER versus WoVR. The $N = 0$ seed-only anchor is always shown on the same plots so readers can see how much of any gain comes from Stage-1 training rather than from the common 120-context warm-start alone.

Matched-budget world-model baseline. The WoVR/RLinf baseline receives the same common seed-warmed starting checkpoint π_{θ_1}, the same shared 120-context seed set, and the same real training budget N as CAVER. In addition, it may use imagined rollouts generated by its own world-model stack, but its real interaction count may never exceed the current budget point N. The imagined-rollout multiplier is fixed before Stage 1 at $M = 32$ imagined candidate rollouts per verified context and is not re-tuned online. The entire baseline is pinned by one immutable local RLinf config file `configs/wovr_piper_h4.yaml`, stored with a SHA256 checksum in `manifest.lock`. That config is derived once from the locked RLinf WoVR-LIBERO release and changes only the study-specific fields: policy initialization path π_{θ_1}, PiPER environment wrapper, chunk horizon $H = 4$, real-budget cap N, rollout horizon 4 chunks, imagined multiplier $M = 32$, one policy-update block after each 25-context round, and exactly 500 optimizer steps per update block. All other RLinf/WoVR fields remain the upstream defaults at the locked commit and may not be changed between methods, budgets, or seeds except for the random seed itself. If the public RLinf/WoVR stack requires more real data than the matched budget to become operational on PiPER, the comparison is reported as "not feasible under the matched-budget protocol" rather than silently given extra real data.

6.4 Baselines and ablations

Primary baselines.

- **Seed-only anchor:** the common seed-warmed checkpoint π_{θ_1} with no Stage-1 training. This baseline isolates the contribution of the shared 120-context warm-start.
- **Real-only baseline:** π-StepNFT on $\pi_{0.5}$ using the same candidate-generation wrapper but a uniform selector over the K candidates, with every executed trajectory admitted to training and no selective-verification correction.
- **World-model-heavy baseline:** WoVR through RLinf, using imagined rollouts as the primary update signal [4, 11].

Mandatory attribution ablations.

- **Selection-only:** keep CAVER's selector and exact-propensity logging, but admit every executed trajectory to StepNFT with no conservative admit/abstain gate.
- **Admission-only:** use the same uniform safe-set selector as the real-only baseline, but apply CAVER's DR calibrator and conservative admit/abstain rule before StepNFT.
- **Random verification:** keep the same conservative gate as full CAVER, but choose the executed safe candidate uniformly at random.
- **Value-only verification:** pick the candidate with the highest raw proxy value and remove uncertainty/diversity/novelty terms from the selector score.
- **No-DR ablation:** keep the selector and admission rule, but replace DR pseudo-outcomes with direct executed-data supervision only.

Mandatory sensitivity analyses.

- **Seed-size curve:** rerun the main $N \in \{0, 25, 50, 100\}$ study with shared seed sizes $N_{\text{seed}} \in \{0, 40, 80, 120\}$ total decision contexts, always reporting both against N and against N_{total}.
- **Frozen-vs-refreshed utility head:** compare the main frozen V_ω against a lightweight refreshed variant that refits V_ω every two rounds on cumulative executed tuples while keeping GE-Sim frozen and keeping selector coefficients fixed.
- **Proxy-target sensitivity:** replace the main $u^{\text{proxy}} = 0.5\, s^{\text{bin}} + 0.5\, q^{\text{prog}}$ target with $u^{\text{proxy}}_\alpha = \alpha s^{\text{bin}} + (1 - \alpha) q^{\text{prog}}$ for $\alpha \in \{0.25, 0.5, 0.75, 1.0\}$.
- **No-provider ablation:** keep the same executed-only backend and selector budget, but remove GE-Sim and use only raw backbone/context features for verification.
- **Weighted-update secondary ablation:** keep the same admitted executed set, but multiply each StepNFT pair by $\max\{0, \text{LCB}_r(z_{r,c,A_{r,c}}) - \tau\}$ before the otherwise unchanged StepNFT loss. This remains secondary and is not part of the main comparison claim.

Fairness rule. All methods share the same public base weights `gs://openpi-assets/checkpoints/pi05_base/params`, the same local `pi05_piper_h4` policy interface, the same seed real dataset, the same common seed-warm-start checkpoint π_{θ_1}, the same deterministic safety shield, the same Stage-0 tasks, the same Stage-1 tasks, the same real-rollout budget, and the same final evaluation protocol. CAVER and the real-only baseline also share the same candidate-generation wrapper and the same executed-update backend: the same $K = 4$ candidate draws per decision context, the same chunk horizon $H = 4$, the same LoRA target modules, the same optimizer, the same learning-rate schedule, the same batch size, the same positive–negative pair construction, the same gradient clip, and the same number of update steps. The real-only baseline uses a uniform selector and admits every executed trajectory to the StepNFT trainer; CAVER differs only by using provider-based scoring, exact-propensity logging, doubly robust calibration, and a conservative admit-or-skip rule before examples enter that same trainer. The mandatory selection-only and admission-only ablations are included precisely so that any observed gain can be attributed more cleanly to verification allocation, conservative admission, or their combination rather than to the unchanged StepNFT backend itself.

6.5 Metrics

The main metrics are:

- **Held-out real-robot success rate:** mean binary success over the 30 final test episodes per task.
- **Success gain per 100 verified contexts:**

$$G_{100} = 100\,\frac{S(200) - S(25)}{200 - 25},$$

  where $S(N)$ is the held-out success rate after training with budget N.
- **Stage-0 selector regret:**

$$\mathrm{Regret}_{\mathrm{S0}} = \frac{1}{C}\sum_{c=1}^{C}\Big(\max_{k\le K} u^{\mathrm{proxy}}_{c,k} - u^{\mathrm{proxy}}_{c,A_c}\Big)$$

  on matched-start simulation contexts with fully known proxy labels.
- **Cross-method calibration metrics:** to make Brier/ECE comparable for CAVER, real-only π-StepNFT, and WoVR, fit one *common post-hoc diagnostic predictor* g_{eval} separately for each method using only that method's executed training data and the same method-agnostic executed feature vector

$$z^{\mathrm{exec}}_{r,c} = [e_{r,c}, \phi_a(a_{r,c,A_{r,c}}), \text{task one-hot}], \qquad g_{\mathrm{eval}}(z) = \sigma(h_\chi(z)),$$

  where h_χ is a 2-layer width-256 GELU MLP and σ is the sigmoid output link. The predictor is trained with binary cross-entropy

$$L_{\mathrm{eval}}(\chi) = -\sum_i \Big(Y_i \log g_{\mathrm{eval}}(z_i) + (1 - Y_i)\log(1 - g_{\mathrm{eval}}(z_i))\Big)$$

  using AdamW with learning rate 10^{-3}, betas $(0.9, 0.999)$, weight decay 10^{-4}, batch size 128, maximum 50 epochs, early stopping patience 5 on fold-validation BCE, and 5-fold cross-fitting. Brier and 10-bin equal-frequency ECE are computed from the resulting out-of-fold probabilities $g_{\mathrm{eval}}(z)$ on validation/test episodes. This is a *common downstream diagnostic*, not each method's native online confidence model. Whenever the paper says "better calibration" in a cross-method table, it refers to this common post-hoc predictor unless the text explicitly says "online CAVER calibrator." The online CAVER calibrator $\bar{\mu}_r(z)$ is reported separately as an internal diagnostic.
- **Interaction efficiency:** wall-clock time and real training interactions required to reach the target success threshold $S^\star = 0.70$ on each task.
- **Deployment latency:** median and 95th-percentile wall-clock latency per decision context, broken into policy sampling, provider rollout/inference, selector/calibrator scoring, and robot idle time waiting for the verification decision.
- **Budget transparency:** every success plot is reported twice, once against matched training budget N and once against total real interaction count N_{total}, and both plots include the seed-only point at $N = 0$.

The continuous progress score $\bar{Y}$ is reported only as a diagnostic.

6.6 A publishability threshold

The project should be judged publishable only if it clears a concrete threshold. The paper is worth writing as a full methods paper if, at $N = 100$, CAVER beats the real-only baseline on at least two of the three real tasks, beats or matches the world-model-heavy baseline on at least two of the three real tasks, clearly exceeds the $N = 0$ seed-only anchor on the task-average success metric, and either (i) improves the *common post-hoc* Brier score or ECE relative to the real-only baseline, or (ii) reaches the target success threshold $S^\star = 0.70$ with fewer real training interactions on at least two of the three tasks. In addition, the mandatory attribution suite must show that the full CAVER configuration outperforms both the selection-only and admission-only ablations on task-average success at $N = 100$, and the seed-size sweep must still show a positive gap over the seed-only anchor when results are plotted against N_{total}. If those conditions are not met, the project should be reframed as an empirical negative-result or systems paper rather than oversold as a new learning method.

7 Concrete implementation plan: Stage-0 LIBERO and Stage-1 PiPER instantiations

This section pins two concrete instantiations of the abstract CAVER interfaces from the main method. Stage 0 is the simulation proxy study: LIBERO acts as the trusted execution environment, a LIBERO-specific execution adapter instantiates Γ_{exec}, a LIBERO-to-GE-Sim bridge instantiates Φ_Ω, and the simulator task-success signal instantiates M_{task}. Stage 1 is the real transfer study: the concrete execution adapter is denoted

$$\Gamma_{\pi \to p} \coloneqq \Gamma_{\text{exec}},$$

and maps raw $\pi_{0.5}$ chunks into PiPER joint/gripper execution commands. The concrete provider adapter is denoted

$$\Phi_{\text{GE}} \coloneqq \Phi_\Omega,$$

and maps those execution commands into the public GE-Sim conditioning format. The concrete safety shield is the PiPER URDF-based hard filter described below, and the concrete verified labeler is the marker-based geometric checker used in the Stage-1 real study. Porting CAVER to another robot changes these implementation modules and the associated local configs, but it does *not* change selector scoring, exact propensity logging, doubly robust correction, calibrator fitting, or the StepNFT-style executed-update backend.

7.1 Concrete Stage-0 LIBERO simulation instantiation

Stage 0 is not just a debugging convenience; it is the concrete simulation instantiation used to fit the proxy head, lock selector defaults, and validate the roundwise protocol before any PiPER run is attempted. Each Stage-0 run uses LIBERO as the trusted execution environment, keeps the same chunk horizon $H = 4$, candidate count $K = 4$, round size $B_{\text{round}} = 25$, and matched counted-context budgets used later, and compares CAVER directly against the real-only π-StepNFT path plus attribution ablations on five LIBERO task families.

The Stage-0 bridge preserves the method-level interfaces rather than introducing a separate algorithm. Raw $\pi_{0.5}$ chunks are converted into LIBERO simulator control commands through a deterministic execution adapter, the same chunks are mapped into GE-Sim-compatible provider arrays through a LIBERO-to-GE-Sim compatibility map, and the trusted label comes from the simulator's task-completion signal together with the fixed Stage-0 progress diagnostics. This makes the Stage-0 study a concrete provider-aware selective-verification experiment in its own right, not a placeholder for the PiPER section.

7.2 Two-machine deployment

Use two machines throughout the project.

- **Robot-control machine (Ubuntu 20.04 + ROS Noetic).** Runs `piper_sdk`, `piper_ros`, camera drivers, the execution bridge, and the reset/checklist UI.
- **Learning machine (Ubuntu 22.04 + GPU).** Runs `openpi`, Genie Envisioner / GE-Sim inference, Stage-0 LIBERO experiments, the DR calibrator, the selector, the replay buffer, and the baseline training code.

This split matches the public software assumptions: openpi documents a `uv` workflow on Ubuntu 22.04, RLinf recommends Docker on Ubuntu 22.04, and the PiPER ROS stack targets ROS Noetic on Ubuntu 20.04 [9, 11, 20].

7.3 Pinned backbone config and software revisions

The proposal now pins the concrete software stack closely enough that the action semantics and baseline interfaces are no longer branch-dependent.

Pinned public revisions and local config names

Artifact	Locked identifier used in this study
openpi repository	commit `fdc03f527881cdfc8ae1a168ed6a20c60edbbbcc`
Backbone weights	`gs://openpi-assets/checkpoints/pi05_base/params`
Local policy config	`pi05_piper_h4` = public `pi05_libero` config with action horizon changed to $H = 4$ and outputs changed to `PiperJointDeltaOutputs`
Genie Envisioner / GE-Sim	commit `0e0de7e2ed8ea04e6ba41d47c93964ae65e5fbd2`
pi-StepNFT backend	commit `140eece2ef7e8574c34c69a221fd4e3f56c2423c`
RLinf / WoVR baseline	commit `e62de7767f41a61d58eb15d916e8809c70610672` and Docker tag `rlinf/rlinf:agentic-rlinf0.2-maniskill_libero`
Local WoVR config	`configs/wovr_piper_h4.yaml`: policy init = π_{θ_1}, PiPER env wrapper, horizon $H = 4$, rollout horizon 4 chunks, imagined multiplier $M = 32$, one update block per 25-context round, 500 optimizer steps per block; SHA256 stored in `manifest.lock`
PiPER ROS stack	`piper_ros` `noetic` branch; exact URDF checksum stored in `manifest.lock` before the first real run

If any public repository changes after these revisions, the study still stays on the locked identifiers above. Before the first real run, the codebase must materialize a `manifest.lock` file containing these exact SHAs, the GE-Sim weight identifier, the PiPER URDF checksum, the SHA256 checksums of the two local config files `pi05_piper_h4.py` and `wovr_piper_h4.yaml`, and the local repository commit.

7.4 Verified setup commands

Backbone and general training stack.

```
mkdir -p robot_posttrain_ws/third_party
cd robot_posttrain_ws/third_party

git clone --recurse-submodules \
  https://github.com/Physical-Intelligence/openpi.git
cd openpi
GIT_LFS_SKIP_SMUDGE=1 uv sync
GIT_LFS_SKIP_SMUDGE=1 uv pip install -e .
```

These commands follow the public openpi setup path [9]. After installation, add the local config `pi05_piper_h4` by copying the public `pi05_libero` config and changing only the action horizon to 4 and the output adapter to `PiperJointDeltaOutputs`; the weight loader remains `gs://openpi-assets/checkpoints/pi05_base/params`.

Frozen provider (Genie Envisioner / GE-Sim).

```
cd /path/to/robot_posttrain_ws/third_party
git clone https://github.com/AgibotTech/Genie-Envisioner.git
conda create -n genie_envisioner python=3.10.4
conda activate genie_envisioner
```

```
cd Genie-Envisioner
pip install -r requirements.txt
# Download GE-Sim weights and required Cosmos2 components
# then run the released inference example on local image/action files.
python gesim_video_gen_examples/infer_gesim.py \
  --config_file=configs/cosmos_model/acwm_cosmos.yaml \
  --image_root=gesim_video_gen_examples/sample_0 \
  --extrinsic_root=gesim_video_gen_examples/sample_0 \
  --intrinsic_root=gesim_video_gen_examples/sample_0 \
  --action_path=gesim_video_gen_examples/sample_0/actions.npy \
  --output_path=gesim_video_gen_examples/sample_0_res
```

These commands match the public GE-Sim inference path released in the repository README [6].

PiPER robot stack.

```
cd /path/to/robot_posttrain_ws/third_party
git clone https://github.com/agilexrobotics/piper_sdk.git
git clone https://github.com/agilexrobotics/piper_ros.git

cd piper_sdk
pip3 install python-can
pip3 install .

cd ../piper_ros
git checkout noetic
source /opt/ros/noetic/setup.bash
catkin_make
bash find_all_can_port.sh
bash can_activate.sh can_piper 1000000 "<USB_PORT>"
```

These steps are taken directly from the public PiPER SDK and PiPER ROS instructions [20, 21].

Camera calibration. Use three fixed RGB cameras rigidly mounted relative to the table and label them `head`, `hand_left`, and `hand_right` to match the GE-Sim config convention. Calibrate intrinsics for each camera with the vendor's checkerboard routine. Calibrate each base-to-camera transform once with a standard hand-eye or checkerboard-in-base-frame routine; if the AgileX hand-eye package is convenient on the installed stack, use that path [22]. Store the resulting matrices in `configs/camera/piper_multiview.yaml`. Because all cameras are static, the same extrinsics are reused across all frames of an episode.

LIBERO Stage 0.

```
conda create -n libero python=3.8.13
conda activate libero
cd /path/to/robot_posttrain_ws/third_party
git clone https://github.com/Lifelong-Robot-Learning/LIBERO.git
cd LIBERO
pip install -r requirements.txt
pip install -e .
python benchmark_scripts/download_libero_datasets.py \
  --datasets libero_object --use-huggingface
```

This follows the public LIBERO repository and dataset instructions [23, 24].

Stage-0 simulation protocol. The concrete Stage-0 study uses the LIBERO adapter above to run the same candidate/chunk protocol that will later be used on PiPER: sample $K = 4$ chunks from $\pi_{0.5}$, score them with the frozen provider and the fixed proxy-plus-calibrator stack, execute at most one safety-approved chunk per counted context, log the exact propensity, and train the unchanged StepNFT-style backend only on admitted executed trajectories. All Stage-0 headline plots compare CAVER against the real-only baseline under matched counted-context budgets on the five LIBERO proxy task families before any Stage-1 transfer claim is made.

World-model-heavy baseline (WoVR through RLinf).

```
cd /path/to/robot_posttrain_ws/third_party
git clone https://github.com/RLinf/RLinf.git
# Follow the Docker-first installation path recommended by RLinf.
docker pull rlinf/rlinf:agentic-rlinf0.2-maniskill_libero
```

RLinf explicitly recommends Docker for embodied RL and explicitly advertises official support for world-model-based VLA RL fine-tuning with WoVR [11].

Real-only baseline backend.

```
cd /path/to/robot_posttrain_ws/third_party
git clone https://github.com/wangst0181/pi-StepNFT.git
```

The released code is the reference backend for both the real-only baseline and the CAVER update layer [10].

7.5 Concrete PiPER–GE-Sim data interface for the current study

The critical concrete integration detail is how sampled policy chunks become *both* PiPER execution commands and GE-Sim conditioning files in the present study. This subsection deliberately pins that interface for the current implementation; other robots would replace only this bridge while leaving the CAVER verification layer unchanged.

Execution-space file. For every candidate, write an `exec_actions.npy` file of shape $H \times 7$ with rows

$$[q_1, q_2, q_3, q_4, q_5, q_6, g].$$

These are the exact absolute joint/gripper targets that will be streamed to PiPER at 10 Hz. This file is the source of truth for real execution.

Provider-space file. From the same execution chunk, deterministically construct

$$\tilde{a}^{\mathrm{GE}}_{r,c,k} = \Phi_{\mathrm{GE}}(a_{r,c,k}) \in \mathbb{R}^{H\times 16},$$

and write it to `actions.npy` in the GE-Sim input folder. Each row has the public dual-arm pose-style layout

$$[x, y, z, q_x, q_y, q_z, q_w, g]_{\mathrm{left}} \parallel [x, y, z, q_x, q_y, q_z, q_w, g]_{\mathrm{right}},$$

where the left arm is the active PiPER arm and the right arm is a fixed parked dummy trajectory.

Camera files. For every context, write synchronized RGB frames for the three fixed views `head`, `hand_left`, and `hand_right`. Store one intrinsics file and one base-to-camera extrinsics file per view. Because all three cameras are static, the same extrinsics are reused across all frames of the episode; the runtime wrapper simply copies the same matrix into the provider input directory for each timestep.

Feature-extraction rule. GE-Sim predicts a sequence of H future frames for every configured view. CAVER always extracts the cheap future embedding from the *final predicted head-view frame* only. The left and right auxiliary views are retained to condition the provider, not to define the final feature.

Week-1 interface test. Before any learning experiment begins, run one interface sanity test on 20 replayed Stage-0 contexts:

1. sample one candidate chunk from $\pi_{0.5}$,
2. write both `exec_actions.npy` and GE-Sim `actions.npy`,
3. verify that PiPER forward kinematics applied to `exec_actions.npy` reproduces the left-arm pose entries written into `actions.npy` to within 5×10^{-3} m and 2°,

4. verify that GE-Sim runs without schema errors and produces an output frame sequence of length H.

If any of these checks fail by the end of week 4, the contingency provider in Appendix D is activated and the paper must say so explicitly.

7.6 Frozen vs trainable modules

What changes online and what never changes in Stage 1

Module	Status in Stage 1	Exact rule
$\pi_{0.5}$ vision/language encoders	Frozen	Used to build context features and start-state embeddings only.
GE-Sim provider $\mathcal{T}_\Omega$	Frozen	Never fine-tuned in the mainline method.
Future-frame encoder F_ρ^{frz}	Frozen	Same feature space for all rounds and all methods.
Proxy head V_ω	Frozen	Trained in Stage 0 + seed data only.
Selector coefficients	Frozen	Tuned once on Stage 0 validation and never learned online.
Calibrator f_{ψ_r}	Updated each round	Refit on all verified data accumulated so far.
Backbone action-head LoRA modules	Updated each round if enough positives exist	Modified only by the unchanged StepNFT loss on admitted executed trajectories.

7.7 Stage-0 tuning grid and Stage-1 defaults

Table 3 lists the fixed hyperparameters and tuning ranges used in the implementation. If Stage-0 validation leaves multiple settings statistically indistinguishable, choose the tie-break default in the rightmost column.

Table 3: Exact tuning grid and tie-break defaults used in the implementation.

Component	Search / fixed value	Tie-break default used if validation is inconclusive
Candidate count K	Fixed at 4	4
Chunk horizon H	Fixed at 4	4
Selector value weight λ_V	$\{1.0, 1.5, 2.0\}$	2.0
Selector uncertainty / disagreement / novelty weights $(\lambda_U, \lambda_D, \lambda_N)$	Each in $\{0, 0.25, 0.5\}$	$(0.25, 0.25, 0.25)$
Selector temperature T	$\{0.25, 0.5, 1.0\}$	0.5
Exploration floor ϵ	$\{0.05, 0.10, 0.20\}$	0.10
Conservative multiplier κ	$\{0.5, 1.0\}$	0.5
Acceptance threshold τ	$\{0, 0.05, 0.10\}$	0.05
Proxy head V_ω	AdamW, lr 10^{-3}, batch 256, 30 epochs max, early-stop patience 5	Same as search setting
Calibrator f_{ψ_r}	AdamW, lr 3×10^{-4}, batch 256, 80 epochs max, early-stop patience 8	Same as search setting
Retrospective DR diagnostic	5-fold cross-fitting with the same 2-layer width-256 MLP family as f_{ψ_r}	5 folds
Action-head adaptation	LoRA rank 16, LoRA scale $\alpha = 32$, dropout 0	Same as search setting
StepNFT backend update	AdamW, lr 10^{-5}, batch 16 positive–negative pairs, max 500 gradient steps per round, gradient clip 1.0	Same as search setting

7.8 Minimal logs that must exist for reproducibility

Before the first real run, write one immutable `manifest.lock` file containing the exact openpi, GE-Sim, RLinf, and pi-StepNFT SHAs; the checkpoint path `gs://openpi-assets/checkpoints/pi05_base/params`; the local config names `pi05_piper_h4` and `wovr_piper_h4.yaml`; the PiPER URDF checksum; the GE-Sim weight identifier; and every study random seed (policy sampling, selector sampling, StepNFT sampling, bootstrap-mask sampling, and train/val/test template shuffles). For every decision context, record run ID, round ID, episode ID, context ID, task ID, and language instruction; all three RGB observation paths, proprioception vector, and camera intrinsics/extrinsics IDs; all execution-space candidate chunks $a_{r,c,1:K}$; all provider-space arrays $\tilde{a}^{\mathrm{GE}}_{r,c,1:K}$; all predicted future-frame file paths from GE-Sim; all cheap features $z_{r,c,1:K}$; selector scores $s_{r,c,1:K}$ and exact propensities $p_{r,c,1:K}$; safety-mask membership $\mathcal{K}^{\mathrm{safe}}_{r,c}$ and any safety reason codes; executed candidate index $A_{r,c}$; binary outcome $Y_{r,c}$, optional diagnostic progress $\bar{Y}_{r,c}$, audit flags, safety-abort flags, and timeout/reset flags; plus model/version hashes for openpi, GE-Sim, RLinf, pi-StepNFT, the PiPER URDF, and the local repository.

7.9 Hard safety shield implementation

The safety shield in Step 4b is implemented on the robot-control machine and is shared unchanged by CAVER, the real-only baseline, the seed-data collection pass, and any WoVR real execution. The bridge evaluates every candidate chunk with the PiPER URDF, the calibrated table plane, and a padded collision model *before* the selector is allowed to sample a real action. The exact rejection rules are the ones in Step 4b: joint bounds, velocity caps, acceleration caps, workspace-box limits, and padded self/table-collision checks. Runtime monitoring then re-checks the currently observed joint state at 20 Hz and triggers the same abort rules during execution. Every rejection or abort is logged with a categorical reason code from the fixed set

{`joint_limit`, `velocity`, `acceleration`, `workspace`, `self_collision`, `table_collision`, `heartbeat`, `tracking`, `manual_est`

The final paper will report the frequency of each reason code per method and per task so that any success gain cannot hide an unsafe execution pattern.

7.10 Real experiment protocol

Each real episode begins from a template object placement defined in `configs/piper_tasks.yaml`. An episode ends when success is reached, when four decision contexts have been used, or when a hard wall-clock timeout of 60 s is reached. Between episodes, the operator performs a fixed reset procedure and checks a three-item list: object placement, camera pose, and gripper state. This reset is not part of the learning algorithm, but it is fixed across all methods to keep the budget comparison fair.

The training/evaluation accounting is also fixed:

- Stage-1 training episodes contribute to the matched budget N only through counted decision contexts.
- Validation and test episodes are logged separately and never mixed into the training buffers or the calibrator fits.
- Any evaluation episode that ends in `safety_abort`, `manual_estop`, or wall-clock timeout is scored as failure ($Y = 0$) for the success metric.
- Human audits are permitted only for end-state labeling when the marker tracker reports low confidence; they are not allowed to modify action choices, StepNFT pair construction, or success thresholds.

7.11 Contingency plan if GE-Sim integration fails early

The proposal should remain executable even if the primary provider is delayed. The contingency trigger is simple: if the team cannot produce stable short-horizon GE-Sim predictions on Stage-0 LIBERO trajectories by the end of week 4, freeze the current CAVER interface and temporarily substitute the lightweight local provider described in Appendix D. That contingency is *not* the main claim and should be clearly labeled as such in the eventual paper.

8 Why this proposal is publishable

This proposal is publishable, if the experiments succeed, because it asks one narrow question, compares against strong matched baselines, and commits to a concrete implementation path. The paper should be framed as a selective-verification study for flow-based VLA post-training under strict matched real budgets, not as a fundamentally new RL algorithm.

Novelty relative to ALOE. ALOE studies action-level off-policy evaluation from logged VLA behavior. CAVER differs by making the data-collection choice itself part of the method: it decides which cheap candidate to verify online, logs exact propensities, and then uses those verified outcomes to decide which executed trajectories deserve a policy update [17].

Novelty relative to WoVR and World-VLA-Loop. WoVR and World-VLA-Loop ask how to use world models more aggressively as simulators or co-evolving partners for RL. CAVER instead treats the world model as a frozen, imperfect signal source and asks how much real verification is needed to make that signal safe enough to exploit [4, 13].

Novelty relative to real-only post-training. Real-only π-StepNFT asks how to update a flow-based VLA from executed trajectories. CAVER asks a different question: given the same real budget and the same executed-update backend, can better selection and correction collect *more informative* executed trajectories [2]? The main experimental burden is therefore attribution, which is why the paper requires the selection-only, admission-only, seed-size, utility-refresh, and proxy-target ablations.

Engineering realism. The implementation uses public stacks that can actually be installed: openpi for $\pi_{0.5}$, GE-Sim for the frozen provider, LIBERO for the concrete Stage-0 simulation study, PiPER SDK/ROS for the concrete Stage-1 execution study, and RLinf for WoVR. The method is written with interface-general interfaces, but the present paper now pins both the simulation-first validation path and the later hardware transfer path so another lab can audit exactly what was run. The proposal also avoids unsupported claims about full online policy-gradient RL on the flow head and instead frames itself as a carefully controlled budgeted post-training study.

Specific future extensions. The most natural next steps are deliberately separated from the present claim. First, replace the executed-sample backend with a πRL-style Flow-SDE backend so that GRPO, PPO, or other true policy-gradient updates become feasible on $\pi_{0.5}$. Second, test whether two or more frozen providers improve robustness through disagreement-aware verification rather than a single GE-Sim predictor. Third, study learned selector policies and explicit safety monitors such as feature-based failure detectors. These are valuable follow-on projects, but excluding them from the main paper keeps the current proposal focused and testable.

9 Conclusion

CAVER asks one focused question that is both scientifically interesting and realistically executable: under the same real verification budget, can selective real execution plus doubly robust correction produce a better post-training signal for a flow-based VLA than either direct real-only updates or more direct trust in imagined world-model experience? The proposal answers that question with an interface-general verification layer defined through an execution adapter, provider adapter, safety shield, and verified labeler, then pins one exact implementation on Stage-0 LIBERO plus a Stage-1 PiPER study. That separation keeps the scientific claim at the interface level while making the present paper concrete enough to implement and audit as a carefully controlled one-robot budget study.

A Concrete operational definitions for the current $\pi_{0.5}$ + GE-Sim + PiPER instantiation

This appendix gathers the implementation choices that must be interpreted literally for the concrete study pinned in the main text. They instantiate the abstract interfaces Γ_{exec}, Φ_Ω, S_{safe}, and M_{task} on the current PiPER hardware stack.

A.1 Concrete PiPER execution semantics

Every executed action chunk is an $H \times 7$ array in PiPER joint space. Rows have the exact layout

$$[q_1, q_2, q_3, q_4, q_5, q_6, g]$$

with $q_{1:6}$ absolute joint targets in radians and g the active gripper command in meters. Commands are streamed at 10 Hz for $H = 4$ steps. Joint values are clipped to the PiPER ROS limits listed in Step 1, and the gripper command is clipped to $[0, 0.035]$.

A.2 Concrete PiPER-to-GE-Sim provider adapter

GE-Sim does not receive the raw $H \times 7$ joint array. It receives the deterministic pose-style file $\tilde{a}^{\text{GE}} = \Phi_{\text{GE}}(a)$ with shape $H \times 16$. The left-arm slots contain the forward-kinematics pose of the PiPER end effector together with the same gripper command; the right-arm slots contain a fixed parked dummy pose and gripper zero. The execution-space and provider-space files are logged side by side for every candidate.

A.3 Concrete constants for the current study

The following constants are interpreted literally in the implementation:

- diversity threshold $\delta_a = 0.05$ under the normalized candidate-distance metric d_{cand};
- parked GE-Sim dummy pose $p^{\text{park}} = [0.80, -0.80, 0.30, 0, 0, 0, 1, 0]$;
- tray containment height margin $h_{\text{tray}} = 0.030$ m above the calibrated tray floor;
- bowl containment height margin $h_{\text{bowl}} = 0.045$ m above the calibrated bowl floor;
- stack planar tolerance $d_{\text{stack}} = 0.020$ m and stack height gap $h_{\text{stack}} = 0.030$ m;
- marker-confidence audit threshold 0.90, with audit triggered if more than 3 of the final 10 frames fall below it.

A.4 Safety shield and abort rules

The deterministic safety shield is part of the reproducible protocol. A candidate chunk is safety-approved only if it passes all five checks in Step 4b. The exact constants are:

- arm velocity caps $\dot{q}^{\text{safe}} = (1.0, 1.0, 1.0, 1.2, 1.2, 1.5)$ rad/s;
- gripper velocity cap $\dot{g}^{\text{safe}} = 0.05$ m/s;
- arm acceleration caps $\ddot{q}^{\text{safe}} = (4, 4, 4, 5, 5, 6)$ rad/s^2;
- gripper acceleration cap $\ddot{g}^{\text{safe}} = 0.25$ m/s^2;
- workspace box $x \in [0.18, 0.55]$ m, $y \in [-0.28, 0.28]$ m, and tool-center height relative to the calibrated table plane $h_{\text{ee}} \in [-0.005, 0.30]$ m;
- self-collision padding margin 8 mm and non-gripper table-penetration tolerance 2 mm;
- runtime abort thresholds: heartbeat loss > 200 ms, observed-vs-commanded joint error > 0.15 rad for > 0.2 s, or any online collision/workspace violation.

For the first command in a candidate chunk, finite-difference safety checks use the *currently observed* robot state $(q_0^{\text{obs}}, g_0^{\text{obs}})$ recorded immediately before the chunk: $\dot{q}_1 = (q_1 - q_0^{\text{obs}})/\Delta t$ and $\dot{g}_1 = (g_1 - g_0^{\text{obs}})/\Delta t$. If the robot API exposes observed joint velocities $(\dot{q}_0^{\text{obs}}, \dot{g}_0^{\text{obs}})$, then $\ddot{q}_1 = (\dot{q}_1 - \dot{q}_0^{\text{obs}})/\Delta t$ and $\ddot{g}_1 = (\dot{g}_1 - \dot{g}_0^{\text{obs}})/\Delta t$; otherwise those initial observed velocities are set to zero. For $t \geq 2$, velocity and acceleration use forward finite differences inside the candidate chunk. If no candidate survives, the bridge issues a 0.4 s hold command, logs `safety_abort`, counts the context toward N, and excludes it from all training tuples. During final evaluation, any episode that terminates with `safety_abort` is scored as failure.

A.5 Exact StepNFT backend

The mirrored-branch StepNFT-style backend is interpreted literally as:

- 51 stored solver states per executed chunk, corresponding to $T_s = 50$ normalized solver steps and $\Delta_s = 1/50$;
- 8 sampled transition tuples per positive–negative pair per epoch: 4 from the positive trajectory and 4 from the negative trajectory, with $t \sim \text{Unif}\{0, \ldots, 49\}$;
- mirrored-branch constant $\beta = 0.05$ and trust-region weight $\lambda_{\text{TR}} = 10^{-4}$;
- one-step moments $\mu_{\theta,t}^{\pm} = x_t + \Delta_s v_\theta^{\pm}(c, x_t, t)$ and $\Sigma_t = 2\Delta_s I$;
- AdamW optimizer with learning rate 10^{-5}, betas $(0.9, 0.999)$, weight decay 10^{-4}, gradient clip 1.0, batch size 16 positive–negative pairs, and exactly 500 gradient steps whenever an update is run;
- the shared seed warm-start pass is the same backend for exactly 500 gradient steps;
- LoRA is attached only to $\{q\_proj, k\_proj, v\_proj, out\_proj, fc1, fc2\}$ in the action-head denoising transformer blocks, with rank 16, scale $\alpha = 32$, and dropout 0.

A.6 Executed-trajectory buffers

After each round, every executed trajectory is placed into exactly one of three cumulative buffers:

- $\mathcal{M}_{\leq r}^{+}$ if $Y = 1$ and LCB $> \tau$,
- $\mathcal{M}_{\leq r}^{-}$ if $Y = 0$ or LCB ≤ 0,
- $\mathcal{M}_{\leq r}^{0}$ if $Y = 1$ and $0 <$ LCB $\leq \tau$.

Only $\mathcal{M}_{\leq r}^{+}$ and $\mathcal{M}_{\leq r}^{-}$ enter the backbone update. $\mathcal{M}_{\leq r}^{0}$ is retained only for calibrator fitting and analysis. Negative reuse across multiple positives is allowed, and pair construction is deterministic under the fixed study seeds.

A.7 Common calibration metric

The main Brier/ECE table compares methods through a common post-hoc diagnostic predictor g_{eval} fitted on executed training data only. The input feature is

$$z^{\text{exec}} = [e, \phi_a(a_{\text{exec}}), \text{task one-hot}], \qquad g_{\text{eval}}(z) = \sigma(h_\chi(z)),$$

where h_χ is a 2-layer width-256 GELU MLP and σ is the sigmoid output link. The training objective is binary cross-entropy

$$L_{\text{eval}}(\chi) = -\sum_i \Big(Y_i \log g_{\text{eval}}(z_i) + (1 - Y_i) \log(1 - g_{\text{eval}}(z_i)) \Big),$$

optimized with AdamW (learning rate 10^{-3}, betas $(0.9, 0.999)$, weight decay 10^{-4}, batch size 128, maximum 50 epochs, early stopping patience 5 on fold-validation BCE) under 5-fold cross-fitting. The reported Brier score and 10-bin equal-frequency ECE are computed from the resulting out-of-fold probabilities. This metric is method-agnostic and therefore valid for CAVER, real-only π-StepNFT, and WoVR. It does *not* measure each method's native online confidence signal; it measures the calibration of a common downstream diagnostic model trained on that method's executed data. CAVER's own online confidence score $\bar{\mu}_r(z)$ is reported separately.

A.8 Pinned revisions

The public software revisions fixed in this proposal are:

- openpi commit `fdc03f527881cdfc8ae1a168ed6a20c60edbbbcc`,
- Genie Envisioner commit `0e0de7e2ed8ea04e6ba41d47c93964ae65e5fbd2`,
- pi-StepNFT commit `140eece2ef7e8574c34c69a221fd4e3f56c2423c`,
- RLinf commit `e62de7767f41a61d58eb15d916e8809c70610672`.

The local policy config name is `pi05_piper_h4` and the local RLinf baseline config is `wovr_piper_h4.yaml`. The exact PiPER URDF checksum, GE-Sim weight identifier, and SHA256 checksums of both local config files are stored in `manifest.lock` before the first real run.

A.9 Seed data and evaluation accounting

The shared seed real set contains 120 decision contexts total, 40 per physical Stage-1 task. It is excluded from the matched-budget training curves but is not hidden: every results table reports both the matched training budget N and the total real interaction count N_{total}, and every main success plot includes the $N = 0$ seed-only anchor point. The seed set first produces the common seed-warmed backbone checkpoint shared by all methods by exactly 500 warm-start gradient steps. CAVER then uses the same seed tuples to fit V_ω and f_{ψ_0}. Stage-1 checkpoints are selected on 10 validation episodes per task and reported on 30 disjoint final test episodes per task. Validation and test episodes are excluded from the training budget and are never used to update the proxy head, calibrator, or backbone.

B Proof sketch for Theorem 5.1

Condition on the history $\mathcal{H}_{r-1}$ and on a particular context c in round r. Because $A_{r,c}$ is sampled from the logged selector with exact propensity $p_{r,c,k}$,

$$\mathbb{E}\left[\frac{\nu(A_{r,c} \mid c)}{p_{r,c,A_{r,c}}} \big(Y_{r,c} - m_{r-1}(z_{r,c,A_{r,c}}) \big) \,\middle|\, c, \mathcal{H}_{r-1} \right] = \sum_{k \in \mathcal{K}^{\text{safe}}_{r,c}} \nu(k \mid c) \big(Y_{r,c}(k) - m_{r-1}(z_{r,c,k}) \big).$$

Adding the model term gives $\sum_{k \in \mathcal{K}^{\text{safe}}_{r,c}} \nu(k \mid c) Y_{r,c}(k)$, and averaging over contexts yields the result.

C Proof sketch for Theorem 5.2

On the event where all confidence intervals are simultaneously valid,

$$U_r(\hat{\nu}_r) \geq \text{LCB}_r(\hat{\nu}_r) \geq \text{UCB}_r(\nu_b) \geq U_r(\nu_b).$$

If no selector satisfies the feasibility constraint, the rule deploys the baseline selector itself and equality follows immediately.

D Contingency-only local provider

This appendix is **not** the main method. It exists only so that the project remains executable if the GE-Sim path is delayed. The contingency is a lightweight local latent provider: use the frozen $\pi_{0.5}$ context embedding, predict one short-horizon latent with a 2-layer MLP, and train a small preview decoder and value head on Stage-0 LIBERO plus seed real data. If this appendix is activated, the paper must state clearly that the mainline off-the-shelf provider failed to integrate on schedule. The scientific selective-verification layer remains unchanged.

References

[1] Shuhan Tan, Kairan Dou, Yue Zhao, and Philipp Krähenbühl. Interactive post-training for vision-language-action models. *arXiv preprint arXiv:2505.17016*, 2025. doi: 10.48550/arXiv.2505.17016. URL https://arxiv.org/abs/2505.17016.

[2] Shuo Wang et al. π-stepnft: Wider space needs finer steps in online rl for flow-based vision-language-action models. *arXiv preprint arXiv:2603.02083*, 2026. doi: 10.48550/arXiv.2603.02083. URL https://arxiv.org/abs/2603.02083.

[3] Kang Chen, Zhihao Liu, Tonghe Zhang, Zhen Guo, Si Xu, Hao Lin, Hongzhi Zang, Xiang Li, Quanlu Zhang, Zhaofei Yu, Guoliang Fan, Tiejun Huang, Yu Wang, and Chao Yu. π_{r1}: Online rl fine-tuning for flow-based vision-language-action models. *arXiv preprint arXiv:2510.25889*, 2025. doi: 10.48550/arXiv.2510.25889. URL https://arxiv.org/abs/2510.25889.

[4] Zhennan Jiang, Shangqing Zhou, Yutong Jiang, Zefang Huang, Mingjie Wei, Yuhui Chen, Tianxing Zhou, Zhen Guo, Hao Lin, Quanlu Zhang, Yu Wang, Haoran Li, Chao Yu, and Dongbin Zhao. Wovr: World models as reliable simulators for post-training vla policies with rl. *arXiv preprint arXiv:2602.13977*, 2026. doi: 10.48550/arXiv.2602.13977. URL https://arxiv.org/abs/2602.13977.

[5] Yue Liao, Pengfei Zhou, Siyuan Huang, Donglin Yang, Shengcong Chen, Yuxin Jiang, Yue Hu, Jingbin Cai, Si Liu, Jianlan Luo, Liliang Chen, Shuicheng Yan, Maoqing Yao, and Guanghui Ren. Genie envisioner: A unified world foundation platform for robotic manipulation. *arXiv preprint arXiv:2508.05635*, 2025. doi: 10.48550/arXiv.2508.05635. URL https://arxiv.org/abs/2508.05635.

[6] AgibotTech. Genie-envisioner repository and ge-sim inference instructions. GitHub repository, 2026. URL https://github.com/AgibotTech/Genie-Envisioner.

[7] Chao Yu, Yuanqing Wang, Zhen Guo, Hao Lin, Si Xu, Hongzhi Zang, Quanlu Zhang, Yongji Wu, Chunyang Zhu, Junhao Hu, et al. Rlinf: Flexible and efficient large-scale reinforcement learning via macro-to-micro flow transformation. *arXiv preprint arXiv:2509.15965*, 2025. URL https://arxiv.org/abs/2509.15965.

[8] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Allen Z. Ren, Lucy Xiaoyang Shi, Laura Smith, Jost Tobias Springenberg, Kyle Stachowicz, James Tanner, Quan Vuong, Homer Walke, Anna Walling, Haohuan Wang, Lili Yu, and Ury Zhilinsky. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. doi: 10.48550/arXiv.2504.16054. URL https://arxiv.org/abs/2504.16054.

[9] Physical Intelligence. openpi. GitHub repository, 2026. URL https://github.com/Physical-Intelligence/openpi.

[10] Shuo Wang et al. π-stepnft code repository. GitHub repository, 2026. URL https://github.com/wangst0181/pi-StepNFT.

[11] RLinf Project. Rlinf documentation and repository. GitHub repository and Read the Docs documentation, 2026. URL https://github.com/RLinf/RLinf.

[12] Dacheng Li, Yunhao Fang, Yukang Chen, Shuo Yang, Shiyi Cao, Justin Wong, Michael Luo, Xiaolong Wang, Hongxu Yin, Joseph E. Gonzalez, Ion Stoica, Song Han, and Yao Lu. Worldmodelbench: Judging video generation models as world models. *arXiv preprint arXiv:2502.20694*, 2025. doi: 10.48550/arXiv.2502.20694. URL https://arxiv.org/abs/2502.20694.

[13] Xiaokang Liu, Zechen Bai, Hai Ci, Kevin Yuchen Ma, and Mike Zheng Shou. World-vla-loop: Closed-loop learning of video world model and vla policy. *arXiv preprint arXiv:2602.06508*, 2026. doi: 10.48550/arXiv.2602.06508. URL https://arxiv.org/abs/2602.06508.

[14] Zengjue Chen, Runliang Niu, He Kong, and Qi Wang. Tgrpo: Fine-tuning vision-language-action model via trajectory-wise group relative policy optimization. *arXiv preprint arXiv:2506.08440*, 2025. URL https://arxiv.org/abs/2506.08440.

[15] Chia-Yu Hung, Navonil Majumder, Haoyuan Deng, Liu Renhang, Yankang Ang, Amir Zadeh, Chuan Li, Dorien Herremans, Ziwei Wang, and Soujanya Poria. Nora-1.5: A vision-language-action model trained using world model- and action-based preference rewards. *arXiv preprint arXiv:2511.14659*, 2025. URL https://arxiv.org/abs/2511.14659.

[16] Hengtao Li, Pengxiang Ding, Runze Suo, Yihao Wang, Zirui Ge, Dongyuan Zang, Kexian Yu, Mingyang Sun, Hongyin Zhang, Donglin Wang, and Weihua Su. Vla-rft: Vision-language-action reinforcement fine-tuning with verified rewards in world simulators. *arXiv preprint arXiv:2510.00406*, 2025. URL https://arxiv.org/abs/2510.00406.

[17] Rushuai Yang, Hecheng Wang, Chiming Liu, Xiaohan Yan, Yunlong Wang, Xuan Du, Shuoyu Yue, Yongcheng Liu, Chuheng Zhang, Lizhe Qi, Yi Chen, Wei Shan, and Maoqing Yao. Aloe: Action-level off-policy evaluation for vision-language-action model post-training. *arXiv preprint arXiv:2602.12691*, 2026. URL https://arxiv.org/abs/2602.12691.

[18] Qiao Gu, Yuanliang Ju, Shengxiang Sun, Igor Gilitschenski, Haruki Nishimura, Masha Itkina, and Florian Shkurti. Safe: Multitask failure detection for vision-language-action models. *arXiv preprint arXiv:2506.09937*, 2025. doi: 10.48550/arXiv.2506.09937. URL https://arxiv.org/abs/2506.09937.

[19] Borong Zhang, Yuhao Zhang, Jiaming Ji, Yingshan Lei, Josef Dai, Yuanpei Chen, and Yaodong Yang. Safevla: Towards safety alignment of vision-language-action model via constrained learning. *arXiv preprint arXiv:2503.03480*, 2025. doi: 10.48550/arXiv.2503.03480. URL https://arxiv.org/abs/2503.03480.

[20] agilexrobotics. piper_ros (noetic branch). GitHub repository, 2026. URL https://github.com/agilexrobotics/piper_ros/tree/noetic. Accessed March 10, 2026.

[21] agilexrobotics. piper_sdk. GitHub repository, 2026. URL https://github.com/agilexrobotics/piper_sdk. Accessed March 10, 2026.

[22] agilexrobotics. handeye_calibration_ros. GitHub repository, 2026. URL https://github.com/agilexrobotics/handeye_calibration_ros. Accessed March 10, 2026.

[23] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/8c3c666820ea055a77726d66fc7d447f-Abstract-Datasets_and_Benchmarks.html.

[24] Lifelong-Robot-Learning Lab. Libero repository. GitHub repository, 2026. URL https://github.com/Lifelong-Robot-Learning/LIBERO.